

IoT and Embedded Systems

Course Instructor: Dr. Dev Narayan Yadav

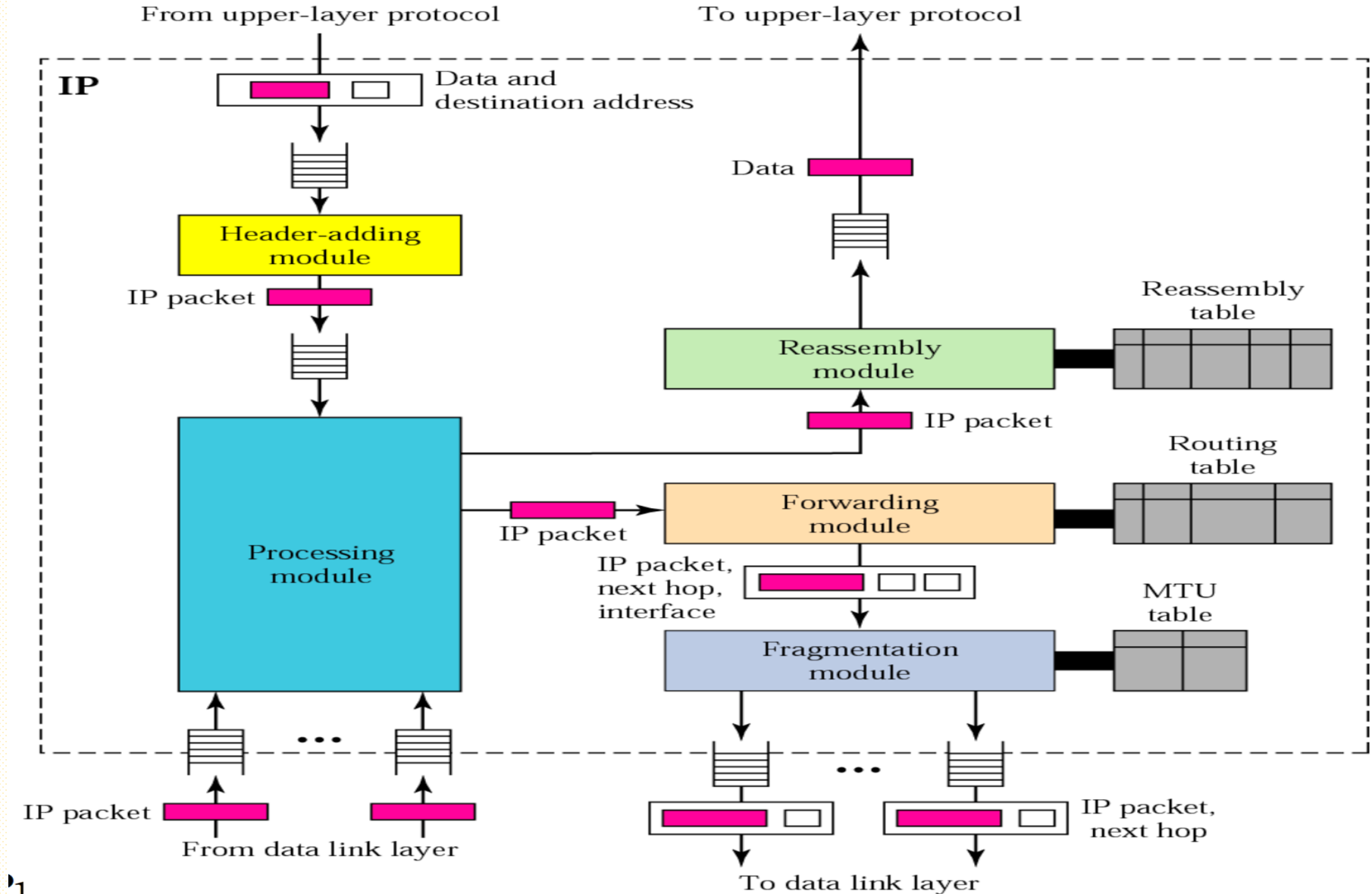
Department of CSE, NIT Rourkela

Email: yadavd@nitrkl.ac.in

Room – CS231

Internet Protocol Design

IP Packet Processing



❑ Packet Generation :

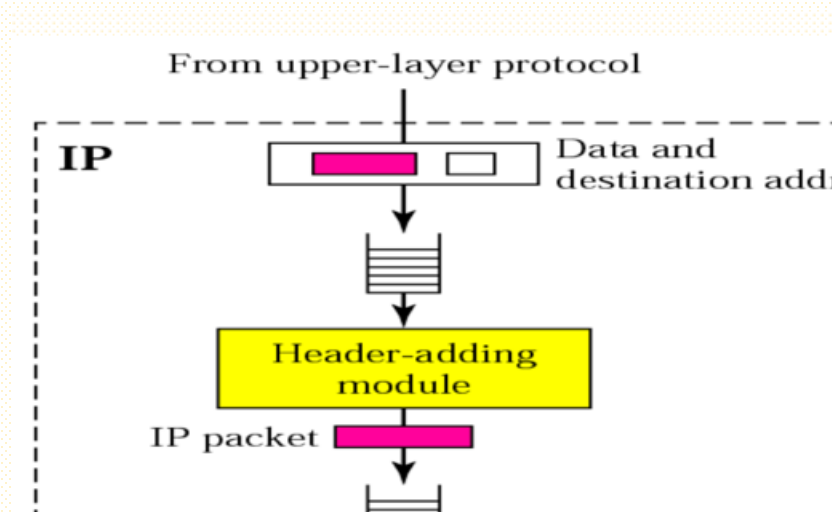
- Upper-layer protocol gives data.
- Header is added → becomes an IP packet.
- Goes through Processing Module.
- Routed by Forwarding Module.
- Fragmented if needed.
- Sent to the Data Link Layer.

❑ Packet Reception:

- Arrives from Data Link Layer.
- Sent to Processing Module.
- If fragmented, reassembled.
- Delivered to upper-layer protocols.

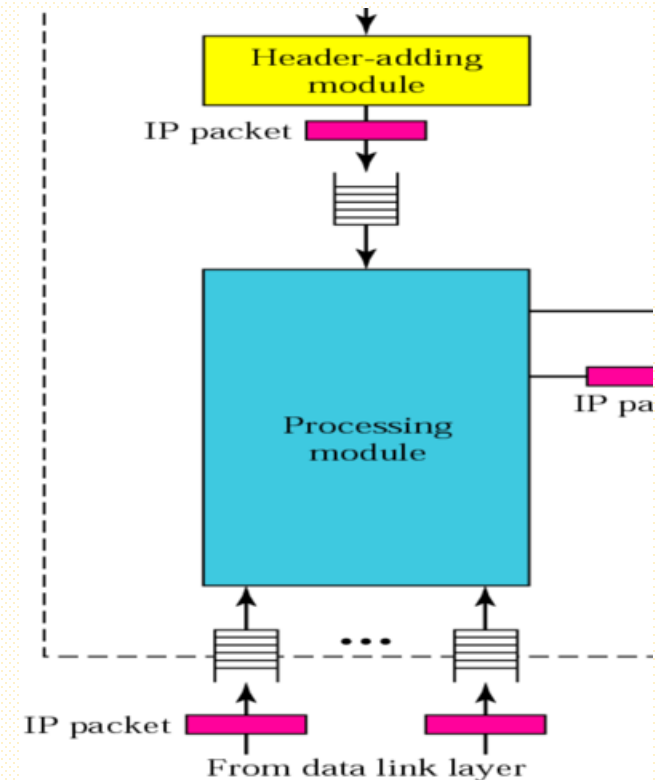
Header Adding Module

- ❑ Encapsulate the data (received from upper layer) in an IP datagram by adding IP headers (i.e. SA, TTL, Version, HLEN, Length, etc.)



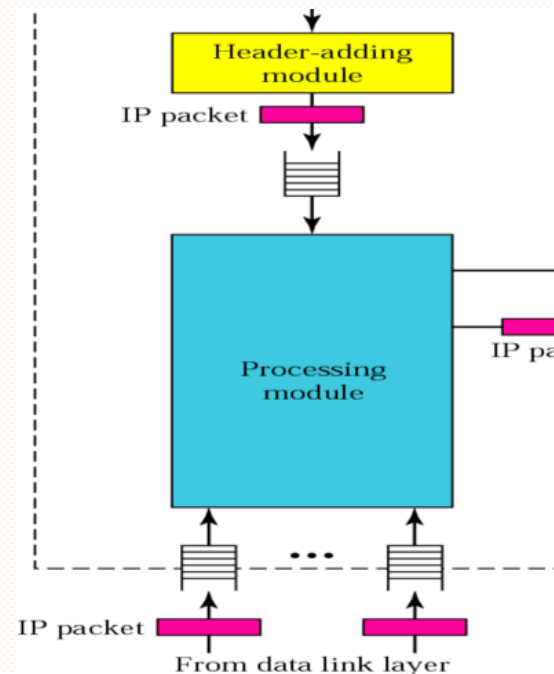
Processing Module

- ❑ The processing module receives the datagram from an Interface or from the Header-adding module and does the following:
 - Select a datagram from one of the input queues.
 - If the destination address matches loopback/ local address, Send the packet to Reassembly module.
 - If the machine is a router,
 - Decrease TTL.
 - If TTL is less than or equal to 0,
 - Discard the datagram.
 - Otherwise, send the datagram to forwarding module



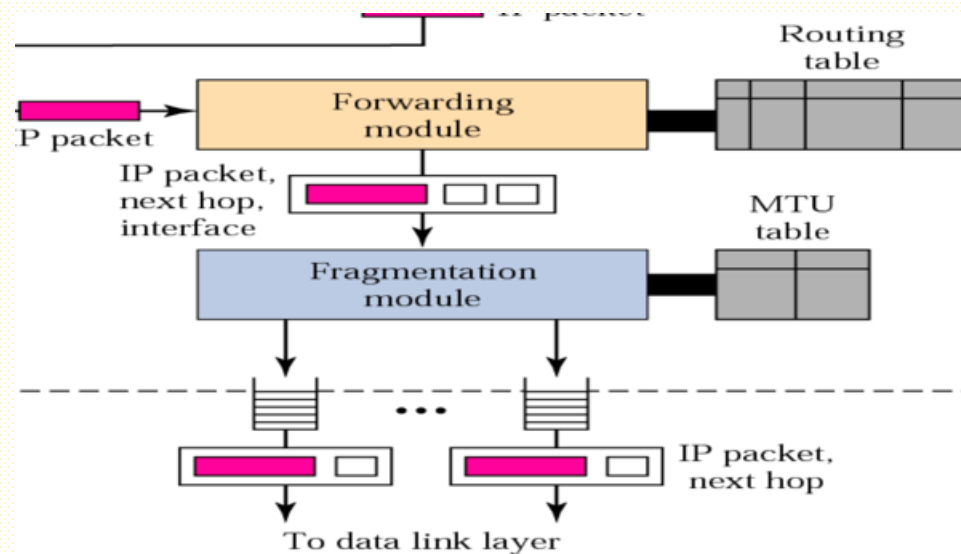
Input/output Queue

- ❑ The input queues store the datagrams coming from the data link layer or the upper layer protocols.
- ❑ The output queues store the datagrams going to the data link layer or the upper layer protocols.
- ❑ The processing module removes datagrams from the input queues.
- ❑ The fragmentation and reassembly modules add the datagram into the output queues.



Forwarding Module

- ❑ The forwarding module receives an IP packet from the processing module.
- ❑ The module finds the IP address of the next-hop along with the interface number to which the packet should be sent.



Routing Table

- ❑ Used by the Forwarding Module to decide the next hop for an IP packet:

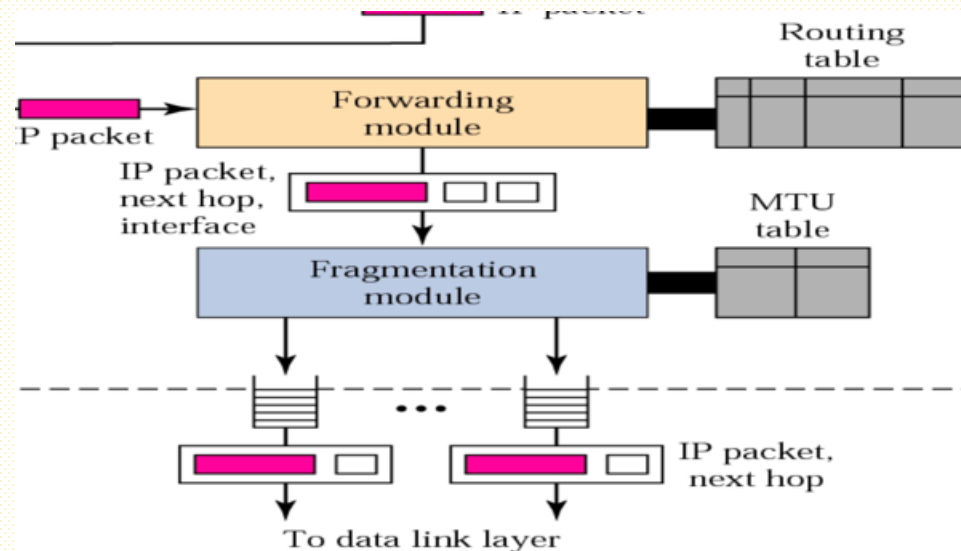
dgimf

Destination	Gateway	NetMask	Flag	Interface	Metric
192.168.1.0	0.0.0.0	255.255.255.0	U	Eth0	1
10.10.0.0	192.168.1.1	255.255.0.0	UG	Eth0	5
172.16.5.20	192.168.1.254	255.255.255.255	UGH	Eth0	10
0.0.0.0	192.168.1.1	0.0.0.0	UG	Eth0	5

- ❑ Destination Network + Mask → determines what IP ranges this route handles.
- ❑ Gateway: is the IP of the next router.
- ❑ Interface: tells which port to send the packet from.
- ❑ Metric helps choose the best route when multiple routes exist.
- ❑ Flag: Indicate type of route.

Fragmentation Module

- ❑ Extract the size of the datagram.
- ❑ If size is greater than the MTU of the corresponding network.
 - If “D” bit is set, a **discard** the datagram.
 - Else, divided the datagram into fragments, add header to each fragment. Send the datagram.
- ❑ Else send the datagram.



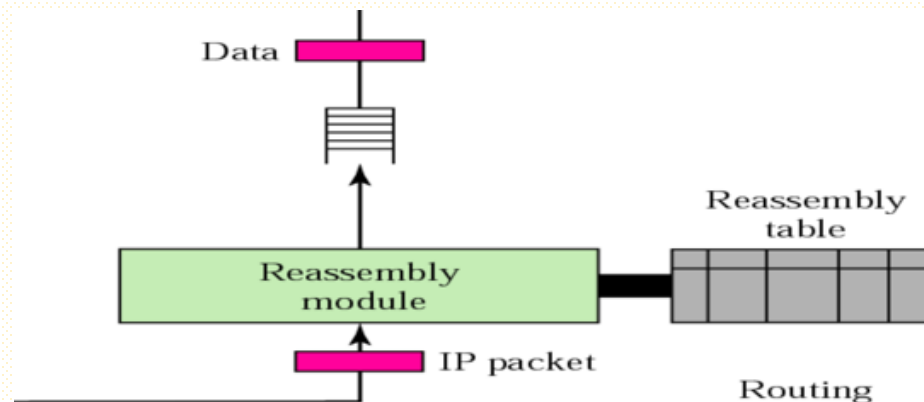
MTU Table

- ❑ The MTU table has two columns Interface number and MTU.
- ❑ The MTU table is used by the fragmentation module to find the maximum transfer unit of a particular interface.
- ❑ Each interface might have a different MTU depending on the link technology.

Interface	MTU Size
Eth0	1500 bytes
Eth1	1400 bytes
Eth2	800 bytes
Wlan0	1200 bytes

Reassembly Module

- ❑ If offset value is 0 and the “more fragment” bit is 0, send the datagram to the appropriate queue.
- ❑ Else search the reassembly table for the corresponding entry.
- ❑ If not found, create a new entry.
- ❑ Insert the fragment at the appropriate place in the linked list.(based on the offset)
- ❑ If all fragments had arrived (“M”-bit and offset), reassembly the fragments, deliver the datagram to the upper layer protocol.
- ❑ Check the T.O., if the T.O. expired discard all fragments.



Reassembly Table



- ❑ Used by the Reassembly Module to track fragmented packets and reassemble them when all fragments arrive:

Source	Destination	Identification	Offset	Status
192.168.1.10	10.0.0.1	0x4A1B	0	Received
192.168.1.10	10.0.0.1	0x4A1B	501	Waiting
192.168.1.10	10.0.0.1	0x4A1B	1000	Received

- ❑ **Identification: ID** for the fragmented packet
- ❑ **Offset: Position** of the fragment in original packet.
- ❑ **Status:** Could be: received, waiting, expired, reassembled.

Packet Analysis

Captured Packet



Capturing from Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
915	46.256517	192.168.0.100	142.251.222.110	TCP	55	[TCP Keep-Alive] 60578 → 4
916	46.351791	142.251.222.110	192.168.0.100	TCP	66	[TCP Keep-Alive ACK] 443 →
917	47.013596	142.250.77.106	192.168.0.100	TCP	60	[TCP Keep-Alive] 443 → 605
918	47.013672	192.168.0.100	142.250.77.106	TCP	54	[TCP Keep-Alive ACK] 60500
919	47.599129	192.168.0.100	104.18.36.169	TCP	55	[TCP Keep-Alive] 60566 → 4
920	47.642095	104.18.36.169	192.168.0.100	TCP	66	[TCP Keep-Alive ACK] 443 →
921	47.717820	142.250.67.42	192.168.0.100	TCP	66	[TCP Keep-Alive] 443 → 605
922	47.718028	192.168.0.100	142.250.67.42	TCP	54	[TCP Keep-Alive ACK] 60501
923	48.029753	192.168.0.100	172.217.24.129	TCP	55	[TCP Keep-Alive] 60564 → 4
924	48.099516	142.250.77.142	192.168.0.100	TCP	60	[TCP Keep-Alive] 443 → 605
925	48.099627	192.168.0.100	142.250.77.142	TCP	54	[TCP Keep-Alive ACK] 60503
926	48.104964	172.217.24.129	192.168.0.100	TCP	66	[TCP Keep-Alive ACK] 443 →
927	48.301541	192.168.0.100	104.18.36.169	TCP	55	[TCP Keep-Alive] 60567 → 4

> Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
> Ethernet II, Src: TpLinkTechno_12:01:df (90:9a:4a:12:01:df), Dst: 08:00:27:00:00:00
> Internet Protocol Version 4, Src: 172.64.151.4, Dst: 192.168.0.100
> Transmission Control Protocol, Src Port: 443, Dst Port: 60567

0000	b8 9a 2a 76 2a 0f 90 9a	4a 12 01 df 08 00 45 00
0010	00 28 00 00 40 00 3c 06	3a 7f ac 40 97 04 c0 a8
0020	00 64 01 bb ec 9a b2 98	24 06 f9 2f 2e b9 50 10
0030	00 13 be 92 00 00 00 00	00 00 00 00

Frame Details

- ✓ Frame 3550: 1294 bytes on wire (10352 bits), 1294 bytes captured (10352 bits) on interface 0
Section number: 1
 - Interface id: 0 (\Device\NPF_{AA90814A-6B32-41BF-AFA7-3AAE38227A5C})
Encapsulation type: Ethernet (1)
Arrival Time: Aug 7, 2025 00:45:25.840147000 India Standard Time
UTC Arrival Time: Aug 6, 2025 19:15:25.840147000 UTC
Epoch Arrival Time: 1754507725.840147000
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 0.000347000 seconds]
[Time delta from previous displayed frame: 0.000347000 seconds]
[Time since reference or first frame: 4.991967000 seconds]
Frame Number: 3550
Frame Length: 1294 bytes (10352 bits)
Capture Length: 1294 bytes (10352 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ipv6:tcp:tls:tls]
[Coloring Rule Name: TCP]
[Coloring Rule String: tcp]
 - Ethernet II, Src: 16:44:72:97:69:32 (16:44:72:97:69:32), Dst: Intel_76:26:32:81:40:80
 - Internet Protocol Version 6, Src: 2404:6800:4002:806::200a, Dst: 2409:4002:806::200a

MAC Information

- ✓ Ethernet II, Src: 16:44:72:97:69:32 (16:44:72:97:69:32), Dst: Intel_76:2a:0f (b8:9a:2a:76:2a:0f)
 - ✓ Destination: Intel_76:2a:0f (b8:9a:2a:76:2a:0f)
 -0. = LG bit: Globally unique address (factory default)
 -0 = IG bit: Individual address (unicast)
 - ✓ Source: 16:44:72:97:69:32 (16:44:72:97:69:32)
 -1. = LG bit: Locally administered address (manual)
 -0 = IG bit: Individual address (unicast)
- Type: IPv6 (0x86dd)
[Stream index: 0]

IPv4 Packet

- ✓ Internet Protocol Version 4, Src: 104.18.32.47, Dst: 192.168.0.100
 - 0100 = Version: 4
 - 0101 = Header Length: 20 bytes (5)
- ✓ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
 - 0000 00.. = Differentiated Services Codepoint: Default (0)
 -00 = Explicit Congestion Notification: Not ECN-Capable Transport
- Total Length: 52
- Identification: 0x0000 (0)
- ✓ 010. = Flags: 0x2, Don't fragment
 - 0... = Reserved bit: Not set
 - .1.. = Don't fragment: Set
 - ..0. = More fragments: Not set
 - ...0 0000 0000 0000 = Fragment Offset: 0
- Time to Live: 60
- Protocol: TCP (6)
- Header Checksum: 0xf576 [validation disabled]
[Header checksum status: Unverified]
- Source Address: 104.18.32.47
- Destination Address: 192.168.0.100
- [Stream index: 2]

IPv6 Packet

- ✓ Internet Protocol Version 6, Src: 2404:6800:4002:806::200a, Dst: 2409:4001:1000:0000:0000:0000:0000:0000
 - 0110 = Version: 6
 - ✓ 1000 0000 = Traffic Class: 0x80 (DSCP: CS0, ECN: Not-ECT)
 - 1000 00.. = Differentiated Services Code Point (DSCP): 0x80
 -00 = Explicit Congestion Notification (ECN): 00
 - 0100 1000 0110 1011 1000 = Flow Label: 0x486b8
 - Payload Length: 1240
 - Next Header: TCP (6)
 - Hop Limit: 121
- ✓ Source Address: 2404:6800:4002:806::200a
 - [Address Space: Global Unicast]
- ✓ Destination Address: 2409:40e2:2002:bc4c:3dbb:9d99:4a22:4df4
 - [Address Space: Global Unicast]
 - [Stream index: 1]

TCP Segment

- Transmission Control Protocol, Src Port: 443, Dst Port: 60525, Seq: 1, Ack:
 - Source Port: 443
 - Destination Port: 60525
 - [Stream index: 66]
 - [Stream Packet Number: 9]
- ✓ [Conversation completeness: Incomplete (12)]
 - ..0. = RST: Absent
 - ...0 = FIN: Absent
 - 1... = Data: Present
 -1.. = ACK: Present
 -0. = SYN-ACK: Absent
 -0 = SYN: Absent
 - [Completeness Flags: ..DA..]
 - [TCP Segment Len: 0]
 - Sequence Number: 1 (relative sequence number)
 - Sequence Number (raw): 3934931228
 - [Next Sequence Number: 1 (relative sequence number)]
 - Acknowledgment Number: 1 (relative ack number)
 - Acknowledgment number (raw): 91754321
 - 1000 = Header Length: 32 bytes (8)
- ✓ Flags: 0x010 (ACK)
 - 000. = Reserved: Not set
 - ...0 = Accurate ECN: Not set

✓ Hypertext Transfer Protocol

```
> POST /userinfo.php HTTP/1.1\r\n
Host: testphp.vulnweb.com\r\n
Connection: keep-alive\r\n
> Content-Length: 22\r\n
Cache-Control: max-age=0\r\n
Origin: http://testphp.vulnweb.com\r\n
Content-Type: application/x-www-form-urlencoded\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/av
Referer: http://testphp.vulnweb.com/login.php\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-IN,en-GB;q=0.9,en-US;q=0.8,en;q=0.7,hi;q=0.6\r\n
\r\n
\[Response in frame: 483\]
\[Full request URI: http://testphp.vulnweb.com/userinfo.php\]
File Data: 22 bytes
```

✓ HTML Form URL Encoded: application/x-www-form-urlencoded

```
> Form item: "uname" = "admin"
> Form item: "pass" = "admin"
```

Encrypted Data



```
0101 .... = Header Length: 20 bytes (5)
> Flags: 0x018 (PSH, ACK)
Window: 1050
[Calculated window size: 268800]
[Window size scaling factor: 256]
Checksum: 0xd0e6 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
> [Timestamps]
> [SEQ/ACK analysis]
TCP payload (1220 bytes)
TCP segment data (801 bytes)
TCP segment data (379 bytes)
> [14 Reassembled TCP Segments (16406 bytes): #3534(965), #3535(1220), #3536(1220), #3537(1220), #3538(1220), #3539(1220), #3540(1220), #3541(1220), #3542(1220), #3543(1220), #3544(1220), #3545(1220), #3546(1220), #3547(1220)]
< Transport Layer Security
  < TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
    Opaque Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 16401
    Encrypted Application Data [...]: 949844334077377dfda0ee405a3c58ab270f
    [Application Data Protocol: Hypertext Transfer Protocol]
  < Transport Layer Security
```

0000	17	03	03	40	11	94	98	44	33	40	77
0010	40	5a	3c	58	ab	27	0f	ae	b5	9b	a6
0020	56	8e	92	63	d6	63	d6	68	85	89	45
0030	f2	a8	35	45	67	a1	1e	25	f3	ef	75
0040	5a	e1	52	c7	0a	3e	b4	a3	89	2a	c7
0050	95	8e	53	27	73	13	34	d0	0b	8a	32
0060	cf	30	3b	b6	2e	c7	65	9a	76	cc	12
0070	ee	e2	45	d6	96	bc	dc	2f	57	e3	75
0080	48	64	80	74	0a	1c	00	b6	38	18	05
0090	27	1f	11	41	cb	d2	cc	e7	30	73	f2
00a0	02	0a	79	22	d7	2e	87	37	ff	c0	1a
00b0	15	7d	8b	b2	24	a0	08	fd	d4	d8	f3
00c0	b1	9f	2e	d4	20	09	ea	9b	f7	38	72
00d0	c0	4d	3c	d7	fb	c6	7e	c3	ca	9b	b5
00e0	53	d2	af	59	7a	79	fd	5c	8c	cc	47
00f0	ca	82	96	eb	c0	36	5d	5f	24	42	06
0100	bd	10	b8	8e	cd	3c	ed	90	4c	94	d3
0110	71	21	cb	64	e7	92	a2	97	0d	5a	02
0120	cf	af	28	b5	35	3a	c8	5d	b6	7e	0a
0130	36	ca	d3	c1	52	21	d1	f6	ed	04	6c
0140	a9	dc	e0	82	88	ea	95	f5	2f	03	a7
0150	f2	80	b5	18	45	dc	a4	87	f3	56	52
0160	78	e8	72	83	de	f7	84	4c	99	1f	1e

IPv4 Needs Upgrade Why?

❑ # of addresses in IPv4 $\rightarrow 2^{32} \sim 4.3$ Billion

- Rapid extension of internet connected devices: PC, smartphone, smart TV, IoT, Sensors, etc.
- In 2011, IANA officially ran out of available IPv4.

❑ NAT: NAT is a technique used to allow multiple devices in a private network to share one public IP address.

- End-to-end connectivity? $\rightarrow$ No (device are hidden behind router)
- Problem for the protocol like: VoIP (Skype, Zoom), P2P (BitTorrent), Online games (CS:GO)
- Why? NAT modifies the packet, and incoming packets (from a stranger peer) are blocked.
- Solution: NAT Traversal technique (STUN, TURN, ICE, UPnP); extra overhead.

NTM

IPv4 Needs Upgrade Why?



❑ Mobility and IoT Expansion:

- IPv4 was never designed for billions of IoT devices.

❑ Security Consideration: NAT: NAT is a technique used to allow multiple devices in a private network to share one public IP address.

- IPv4: Security features like IPSec are optional.
- IPv6: IPSec is mandatory, enabling encrypted communication at the network layer.

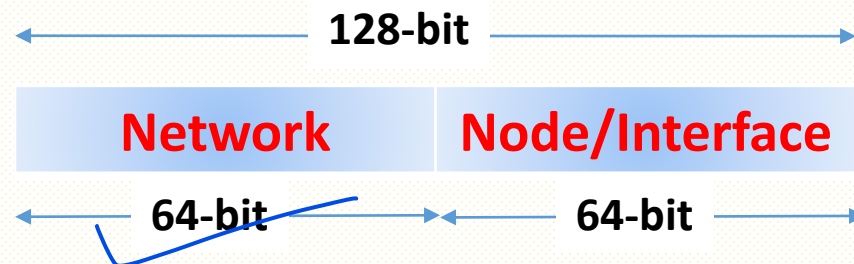
IPv6 Fundamentals & Addressing

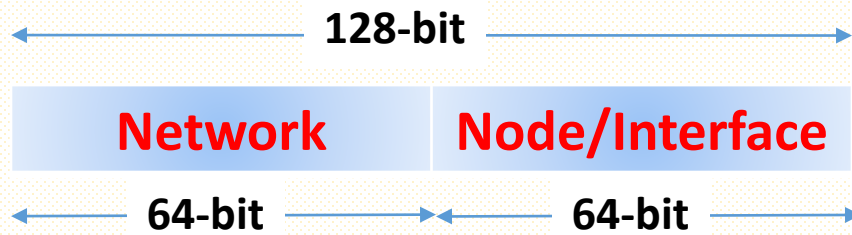
IPv6 (Internet Protocol version 6)



❑ Key Characteristics:

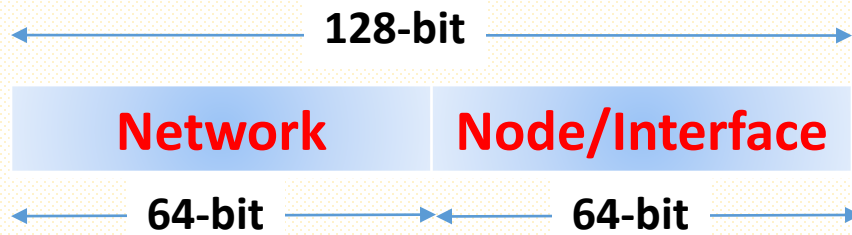
- 128 bit address xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx.
- Where each x is a hexadecimal digit representing 4 bits (16 bits in each block), separated by colons.
- Range:
 - 0000:0000:0000:0000:0000:0000:0000:0000 to
 - FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF
- $2^{128} \rightarrow$ 340 undecillion addresses.
- Example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334
2001:db8:85a3::8a2e:370:7334





- ❑ Network Component: The first 64 bits of the address → used for routing.
- ❑ It is further divided into two parts:
 - Global Routing Prefix (48-bits): Assigned by ISP → used for routing.
 - Subnet ID (16-bit): managed internally → used for sub-netting.
- **2001:0db8:85a3:0000:0000:8a2e:0370:7334**

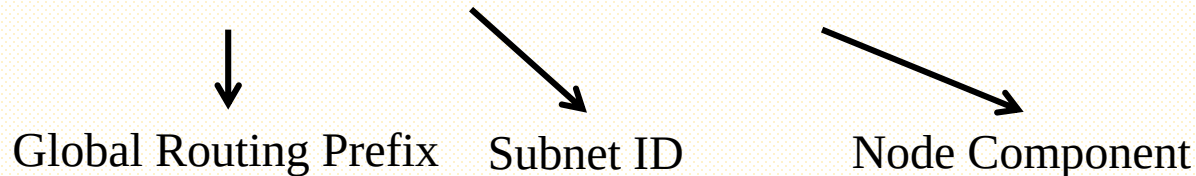
↓ ↘ ↘
Global Routing Prefix Subnet ID Node Component
ISP



❑ Node Component: The latter 64-bits of the address → used to identify the interface.

- Uniquely identifies the interface (host/device)
- It can be Auto-generated using MAC address (EUI-64)
- Randomized for privacy (privacy extensions)
- Manually configured

• **2001:0db8:85a3:0000:0000:8a2e:0370:7334**



Rules for Writing IPv6 Addresses



☐ Omit leading zeros:

☐ Drop any leading 0 in a block.

☐ 0db8 → db8

☐ 0042 → 42

☐ 2001:0db8:0000:0000:0000:ff00:0042:8329 → 2001:db8:0:0:0:ff00:42:8329

☐ Double colon Compression (::)

☐ Specify IPv6 addresses by using double colons (::) in place of a series of zeros.

☐ 2001:db8:0:0:0:ff00:42:8329 → 2001:db8::ff00:42:8329

☐ You can only use one (::) in an IPv6 address. **(Why?)**

Prefixes in IPv6



- ❑ A prefix in IPv6 is the network portion of an address, just like a ZIP code in postal addresses.
 - It defines: Which network the address belongs to.
 - Where routing decisions are made.
 - Where the host (interface) part begins.

- ❑ CIDR Notation: An IPv6 address is often written with a prefix length, just like in IPv4:
 - 2001:0db8:abcd:0012::/64
 - 2001:0db8:abcd:0012:: → network part (first 64 bits).
 - /64 → prefix length (number of bits used for the network).

❑ Common Prefix:

- /128 → Single host (like a loopback)
- /64 → Standard subnet (default in IPv6)
- /48 → Global routing prefix (ISP assigned)
- /32 → Assigned to ISPs by RIRs

❑ Why Is This Important?

- Routers use prefixes to route packets to the correct network.
- Network admins use prefixes to define subnets and allocate address blocks.
- Devices use the prefix to generate interface IDs.



Prefixes in IPv6



- ❑ Let the IPV6 address be:001:db8:3c4d:0015:0000:0000:1a2f:1a2b
- ❑ Zeros compressed → **2001:db8:3c4d::/48** (site prefix)
- ❑ Site Prefix: It identifies an organization's entire network block.
The /48 means the first 48 bits are fixed and represent the network assigned to the organization.
- ❑ Subnet prefix: **2001:db8:3c4d:15::/64** (Starts with 48-bit site prefix, and then 16-bit subnet ID)
- ❑ Special prefixes:
 - ❑ **2002::/16** → IPv6 over IPv4, these addresses help send IPv6 data inside IPv4 packet.
 - ❑ **fe80::/10** → link-local address, local-only addresses, used to talk to devices on the same network or link follows (they don't go through routers)
 - ❑ **ff00::/8** – Multicast address, used to send message to many devices

IPv4 and IPv6 Embedding

- ❑ IPv4-IPv6 embedding means putting an IPv4 address inside an IPv6 address.
- ❑ Why? → Because the internet is slowly moving from IPv4 to IPv6, but many devices still use IPv4.
- ❑ How it is done?
 - ❑ Combines the colon and dotted notation, so the IPv4 address can be embedded in the IPv6 address.
 - ❑ Hexadecimal values are specified for the left-most 96 bits, and decimal values are specified for the right-most 32 bits indicating the embedded IPv4 address.
 - ❑ Embedding ensures compatibility between IPv6 nodes and IPv4 nodes when you are working in a mixed network environment.

Embedding Types

❑ IPv4-Mapped IPv6 Address: Used by IPv6-only systems to communicate with IPv4.

- `::ffff:<IPv4-address>`
- `::ffff:192.168.1.1` → IPv6 version of 192.168.1.1

❑ 6to4 Address (2002::/16): Used to send IPv6 traffic over IPv4 infrastructure.

- `2002:<IPv4-in-hex>::/48`
- IPv4: 192.0.2.4 → Hex: C000:0204
- 6to4 Address: 2002:C000:0204::/48

IPv6 Address Types



❑ IPv6 addresses are categorized into these basic types:

- Unicast address
- Anycast address
- Multicast address

Unicast Address

- ❑ The unicast address specifies a single interface.
- ❑ Used for one-to-one communication → packets are delivered to a single unique interface.
- ❑ Subtypes of Unicast Address:
 - Link-local address:
 - Used for local communication between nodes on the same link (not routable)
 - Link-local addresses are automatically configured on all interfaces. The prefix used for a link-local address is FE80::/10.
 - ✓ • Routers do not forward packets with a destination or source address containing a link-local address.

❑ Subtypes of Unicast Address:

- **Global Unicast**
 - Used for public internet routing.
 - The prefix used for global unicast address is (2000::/3)
- **Unique Local**
- Private or internal communication within an organization..
- The prefix used for unique local address is (FC00::/7)

Special Unicast Address



❑ Unspecified address

- The unspecified address is $0:0:0:0:0:0:0:0$ ($::$).
- The unspecified address indicates the absence of an address, and it can never be assigned to a host.
- It can be used by an IPv6 host that does not yet have an address assigned to it.
- For example, when the host sends a packet to discover if an address is used by another node, the host uses the unspecified address as its source address.

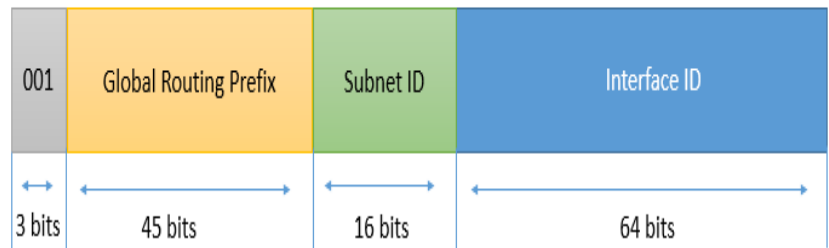
❑ Loopback address

- The loopback address is $0:0:0:0:0:0:0:1$ ($::1$).
- The loopback address is used by a node to send a packet to itself.

Link Local Unicast Address

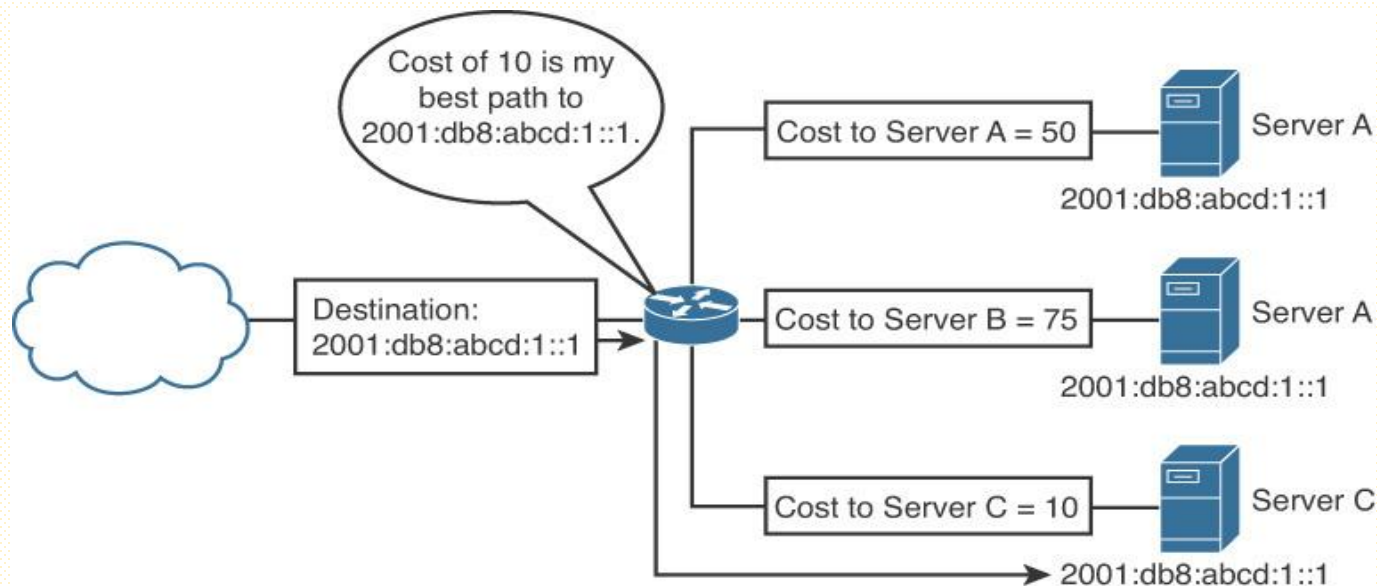


Global Unicast address



Anycast

- ❑ An Anycast address specifies a set of interfaces, possibly at different locations, that all share a single address.
- ❑ A packet sent to an Anycast address goes only to the nearest member of the Anycast group.
- ❑ Used for: Load balancing, CDN nodes (e.g., Google DNS 2001:4860:4860::8888 may be served from a node near you).



Multicast

- ❑ The multicast address specifies a set of interfaces, possibly at multiple locations.
- ❑ Used for one-to-many communication, if a packet is sent to a multicast address, one copy of the packet is delivered to each member of the group.
- ❑ The multicast address replaces broadcast.
 - Neighbor Discovery Protocol
 - Router Advertisement
 - Multicast Listener Discovery.
- ❑ Prefix (8) + Flag (4) + Scope(4) + GroupID(112)

PFSG

Multicast

- ❑ FF: Multicast prefix (11111111)
- ❑ Flags: Additional info like permanent/dynamic
- ❑ Scope: Determines how far the multicast packet travels (interface-local, link-local, global etc.)
 - 0001: Interface local
 - 0010: Link local
 - 0101: Site local
 - 1110: Global
 - E.g. FF02:: → Link Local Scope (sent to all routers on link)
 - E.g. FF01:: → All IPv6 enabled device on link.
- ❑ Group ID: Identifies the group

LLSG

IPv6 Neighbour Discovery

- ❑ Neighbor discovery allows hosts and routers to communicate with one another.
- ❑ Neighbor discovery functions are used by IPv6 nodes (hosts or routers)
 - to discover the presence of other IPv6 nodes,
 - to determine the link-layer addresses of nodes,
 - to find routers that are capable of forwarding IPv6 packets, and
 - to maintain a cache of active IPv6 neighbors

IPv6 Neighbour Discovery

❑ IPv6 nodes use these five Internet Control Message Protocol version 6 (ICMPv6) messages to communicate with other nodes:

- Router solicitation
- Router advertisement
- Neighbour solicitation
- Neighbour advertisement
- Redirect

S A S P R

❑ Router solicitation

- Hosts send these messages to request routers to generate router advertisements.
- A host sends an initial router solicitation when the host first becomes available on the network.

❑ Router advertisement

- Routers send these messages either periodically or in response to a router solicitation.
- The information provided by router advertisements is used by hosts to automatically create global interfaces, and associated routes.
- Router advertisements also contain other configuration information used by a host such as MTU and hop limit.

IPv6 Neighbour Discovery

❑ Neighbor solicitation

- Nodes send these messages to determine the link-layer address of a neighbor, or to verify that a neighbor is still reachable.

❑ Neighbor advertisement

- Nodes send these messages in response to a neighbor solicitation or as an unsolicited message to announce an address change.

❑ Redirect

- Routers use these messages to inform hosts of a better first hop for a destination.

IPv4 vs IPv6



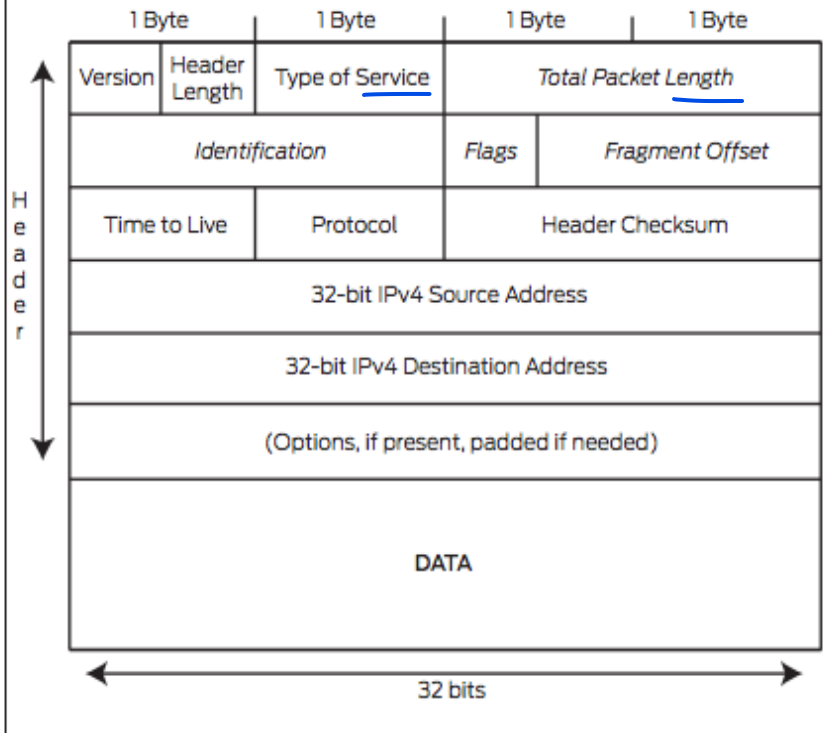
Feature	IPv4	IPv6
Address Size	32-bit	128-bit
Address Format	Decimal (e.g., 192.168.1.1)	Hexadecimal (e.g., 2001:0db8:85a3::8a2e:370:7334)
Header Complexity	Complex, variable length (20– <u>60 bytes</u>)	Simplified, fixed length (<u>40 bytes</u>)
Configuration	Manual or via DHCP	SLAAC (autoconfig), DHCPv6
Security (<u>IPSec</u>)	Optional	Mandatory
Broadcast Support	<u>Yes</u>	No broadcast – uses <u>multicast/anycast</u>
<u>Mobility Support</u>	Limited	Built-in support
Routing Efficiency	Less efficient due to NAT	More efficient

IPv6 Packet

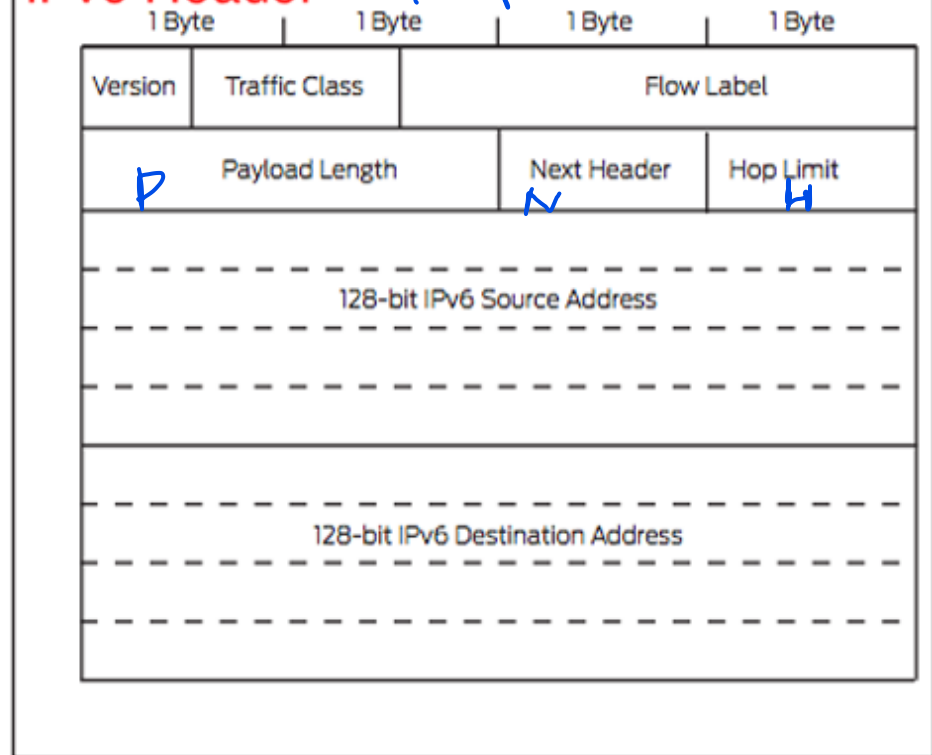
- ❑ The smallest message entity exchanged using Internet Protocol version 6 (IPv6).
- ❑ Packets:
 - Control Information (addressing + routing): mandatory fixed and optional extension headers.
 - Payload: (higher-level datagram or data for internet (ICMPv6) or link layer.
- ❑ In contrast to IPv4, routers do not fragment IPv6 packets larger than the MTU, it is the sole responsibility of the originating node.
- ❑ A minimum MTU of 1,280 bytes is mandated by IPv6, but hosts are "strongly recommended" to use Path MTU Discovery to take advantage of MTUs greater than the minimum.

IPv4 vs IPv6 Header

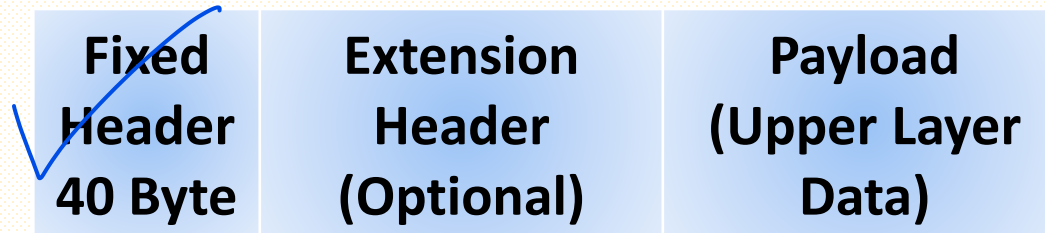
IPv4 Header



IPv6 Header



- ❑ Every IPv6 packet starts with a 40-byte base header. Everything else is optional or belongs to the payload.



IPv6: Fixed Header



- ❑ Every IPv6 packet starts with a 40-byte base header. Everything else is optional or belongs to the payload.

Version (4-bit)	Traffic Class (6+2)-bit	Flow Label	
Payload Length		Next Header	Hop Limit
Source Address (128-bit)			
Destination Address (128-bit)			

IPv6: Fixed Header

- ❑ Version (4 bits):
 - Always set to 6 (0110)
- ❑ Traffic Class (6+2 bits): used for QoS/Priority
 - The six most-significant bits hold the differentiated services field which is used to classify packets (what services should be provided to packet). Replaces ToS field of IPV4.
 - The remaining two bits are used for Explicit Congestion Notification (ECN); priority values subdivide into ranges: traffic where the source provides congestion control and non-congestion control traffic.



IPv6: Fixed Header

❑ Flow Label (20 bits)

- This is used to identify the sequence of packets (packets in same flow). It helps in prioritizing packet delivery and providing real time service.
- Flow: sequence from same source → destination with same traffic class and flow label.
- Flow label 0 means the packet does not belong to any flow.
- Else it identifies flow by SA and port, DA and port and protocol (value of the last Next Header field).

❑ Payload Length (16 bits)

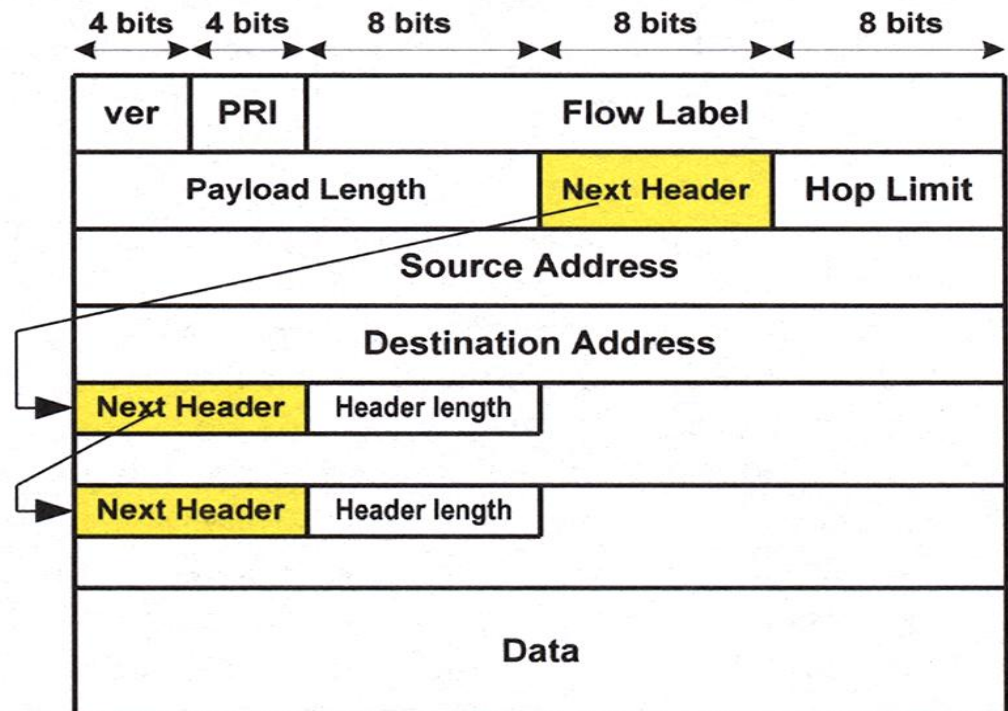
- The size of the payload in bytes, including any extension headers (excluding base header).
- Max → 65535 bytes.
- It is used in place of TLEN of IPv4.

IPv6: Fixed Header

❑ Next Header (8 bits)

- Specifies the type of the extension header that follows the primary IPv6 header. This field usually specifies the transport layer protocol used by a packet's payload. When extension headers are present in the packet this field indicates which extension header follows.

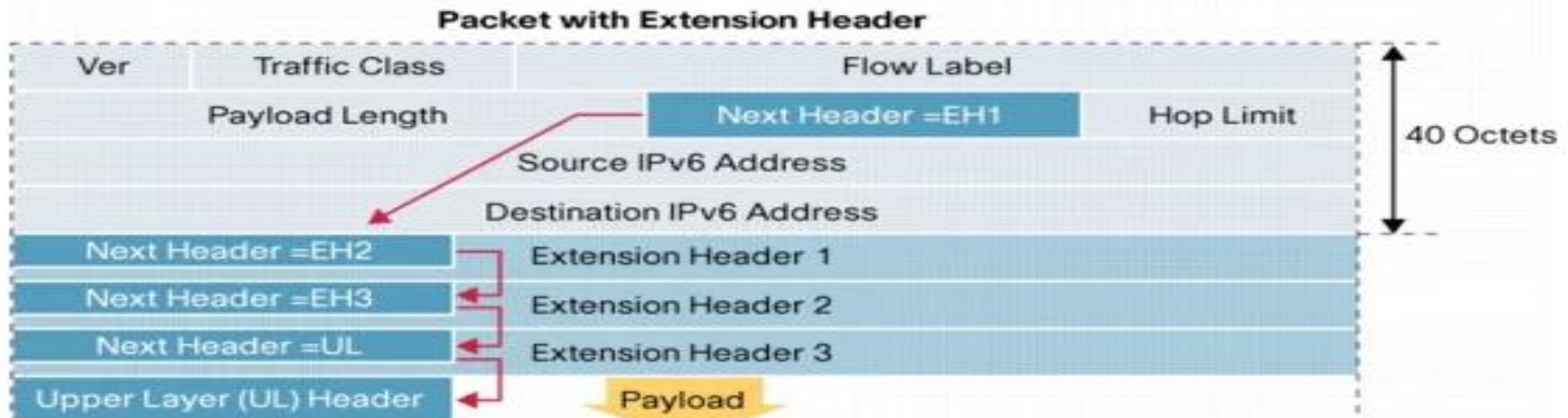
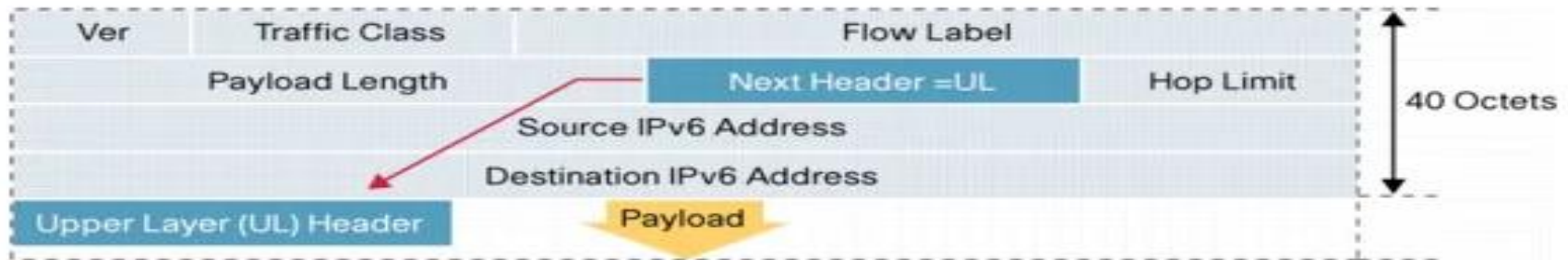
- 6 → TCP
- 17 → UDP
- 58 → ICMPv6
- 59 → No next header



IPv6: Extension Header

❑ Next Header (8 bits)

- This is similar to the protocol field in IPv4 header. Specifies the type of the extension header that follows the primary IPv6 header. This field usually specifies the transport layer protocol used by a packet's payload. When extension headers are present in the packet this field indicates which extension header follows.



IPv6: Fixed Header

❑ Hop Limit (8 bits)

- TTL field in IPv4
- This value is decremented by one at each forwarding node and the packet is discarded if it becomes 0.
- However, the destination node should process the packet normally even if received with a hop limit of 0.

❑ Source/Destination Address (128-128 bits)

- As in IPv4, this specifies the sender's/Receivers IP 128-bit address.

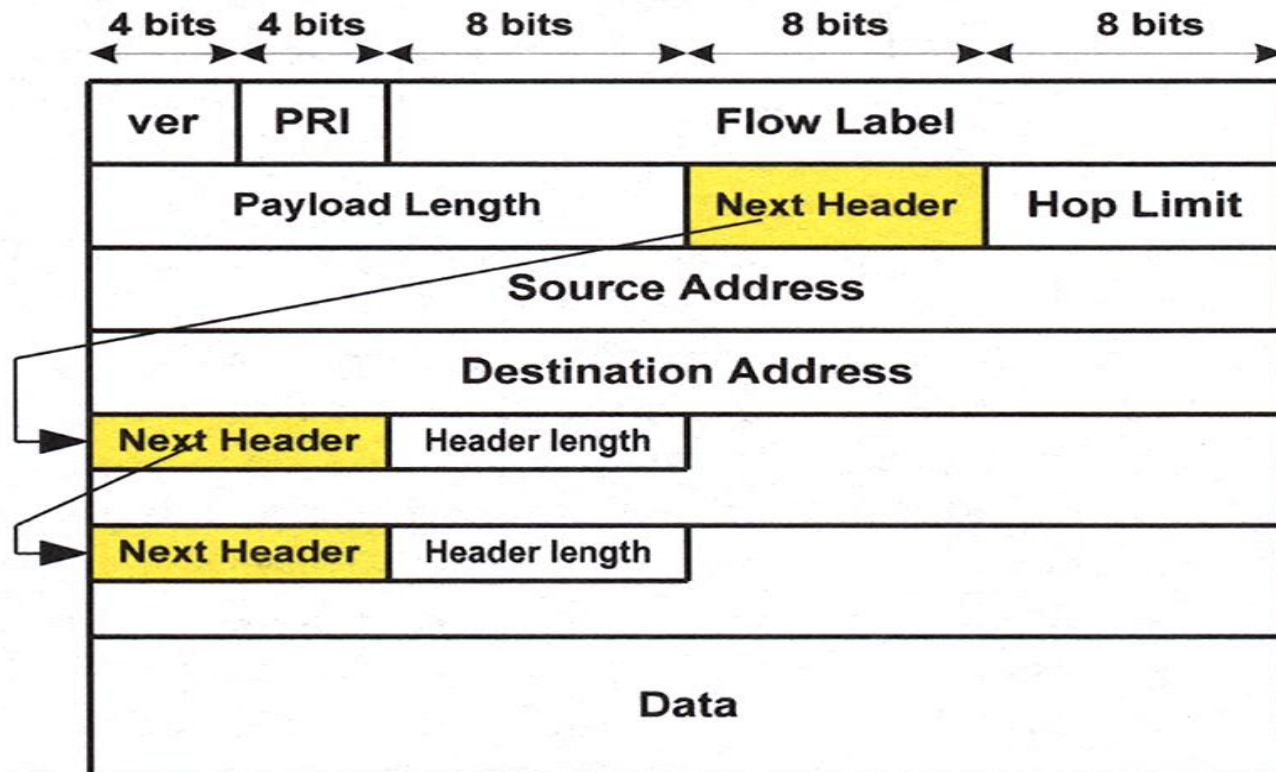
IPv6: Extension Header

- ❑ Extension Headers are optional headers placed between the IPv6 base header and the upper-layer protocol data, used to support additional functionality without increasing the base header size.
- ❑ Must be processed in order as they appear. Next Header field in each header points to the next. (forms a chain, using the Next Header fields)
- ❑ Types:
 - Hop-by-Hop
 - Destination Options (BR)
 - Routing
 - Fragment
 - AH
 - ESP
 - Destination Options (BUL)



IPv6: Extension Header

- ❑ The Next Header field in the fixed header indicates the type of the first extension header; the Next Header field of the last extension header indicates the type of the upper-layer protocol header in the payload of the packet.



IPv6: Extension Header

- ❑ IPv6 packet may contain zero, one or more extension headers but these should be present in their recommended order:

Order	Header Type	Next Header Code
1	Basic IPv6 Header	-
2	Hop-by-Hop Options	0
3	Destination Options (with Routing Options)	60
4	Routing Header	43
5	Fragment Header	44
6	Authentication Header	51
7	Encapsulation Security Payload Header	50
8	Destination Options	60
9	Mobility Header	135
	No next header	59
Upper Layer	TCP	6
Upper Layer	UDP	17
Upper Layer	ICMPv6	58

IPv6: *Extension Header*

- ❑ There are several extension headers defined, and new extension headers may be defined in the future.
- ❑ Most extension headers are examined and processed at the packet's destination.
- ❑ All other extension headers are optional and should only appear at most once, except for the Destination Options header extension, which may appear twice.

IPv6: Extension Header

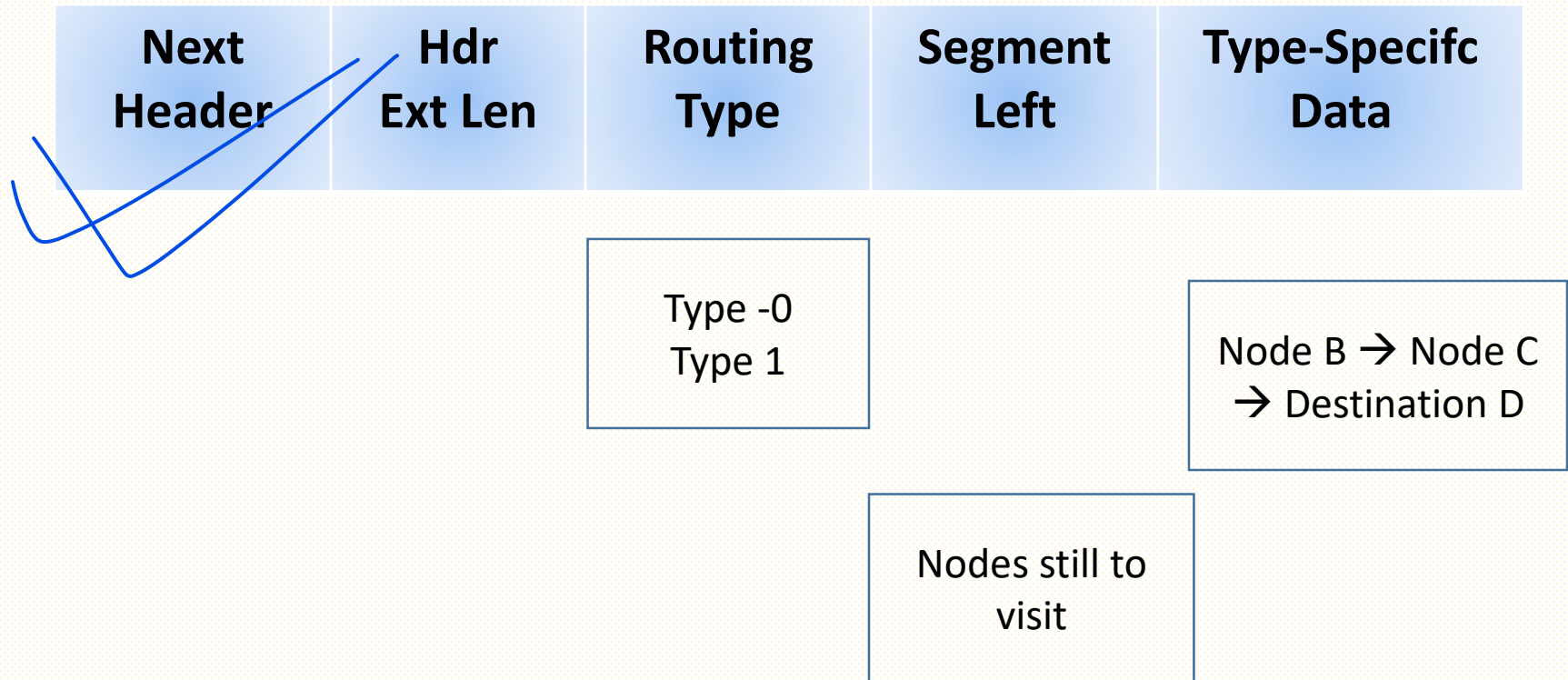
- ❑ Value 59 (No Next Header) in the Next Header field indicates that there is no next header whatsoever following this one, not even a header of an upper-layer protocol.
- ❑ It means that, from the header's point of view, the IPv6 packet ends right after it: the payload should be empty.
- ❑ There could, still be data in the payload if the payload length in the first header of the packet is greater than the length of all extension headers in the packet.
- ❑ This data should be ignored by hosts, but passed unaltered by routers.

Extension Header: Hop-by-Hop

- ❑ The Hop-by-Hop Options extension header may be examined and altered by all nodes on the packet's path, including sending and receiving nodes
- ❑ Hop-by-Hop Options can be processed and modified by intermediate nodes and, if present, must be the first extension.
- ❑ Common use:
 - Jumbo Payload: allows Payload > 65535 byte
 - Router Alert: Instructs routers to examine packet closely.

Extension Header: Routing

- ❑ The Routing extension header is used to direct a packet to one or more intermediate nodes before being sent to its destination.
- ❑ Specifies a set of routers that the datagram must visit.

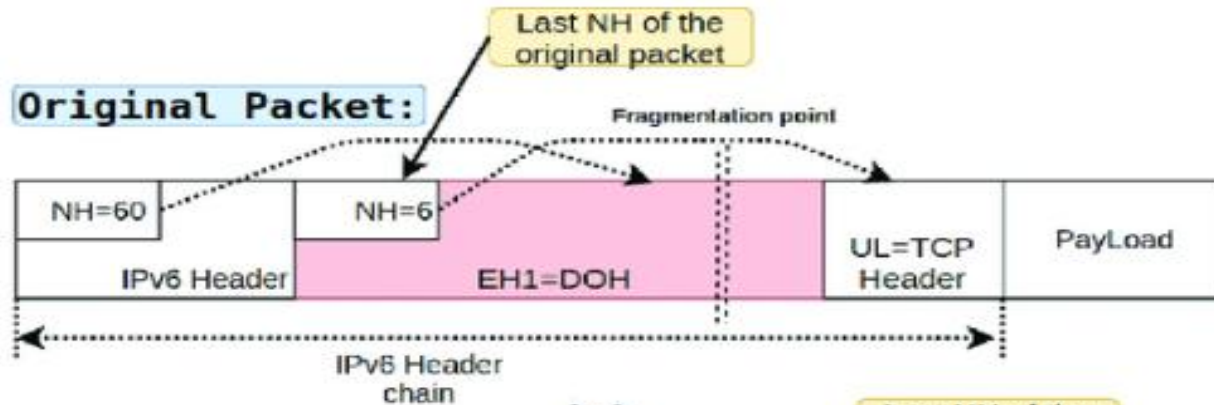


Extension Header: Fragment

- ❑ In IPv6, the routers do not fragment packets. When a packet is too long for a router to handle the router sends an ICMP message to the source asking it to fragment the packet and resend it.
- ❑ This header specifies which fragment of a larger packet this IPv6 packet contains. By using this extension header, the destination can reassemble the packet as in IPv4.

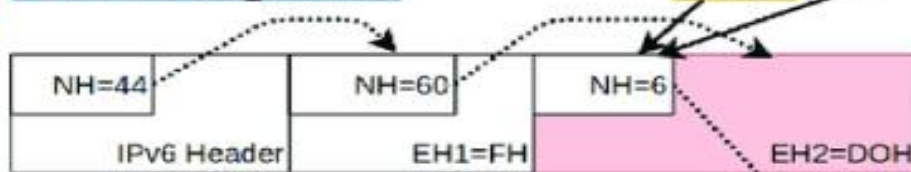
Next Header	Reserved	Fragment Offset	Res	M flag	Identification
		Position in original packet		More fragment	Same for all fragments of a packet

IPv6: Example of Fragment

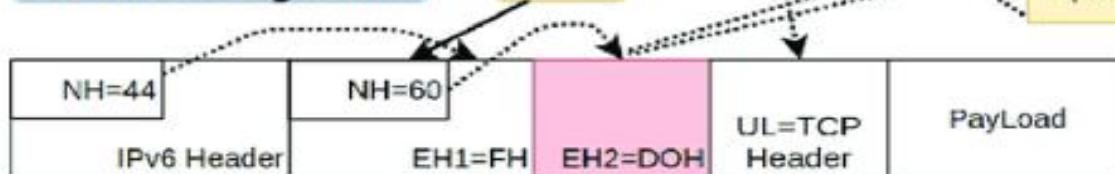


(a)

First Fragment:



Second Fragment:



(b)

NH	Description
60	Dst. Option Hdr.
6	TCP Hdr.
44	Fragmentation Hdr.

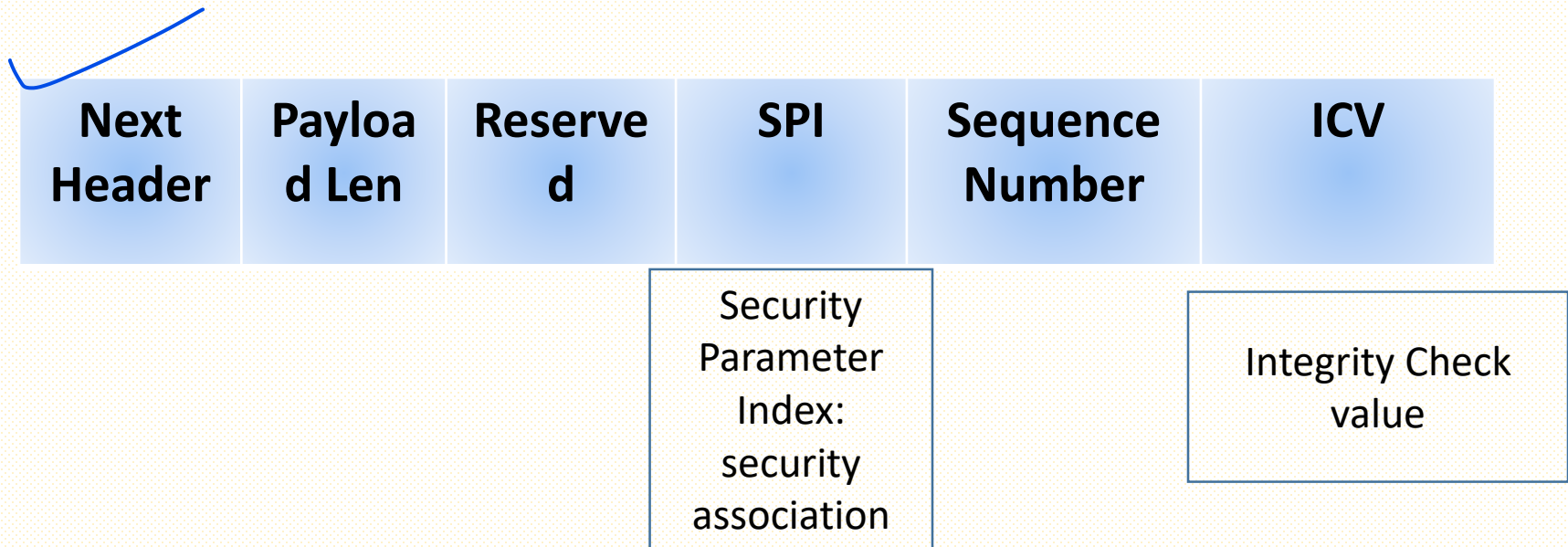
Note that, DOH breaks the header chain by occupying space between FH and ULH

Extension Header: Destination

- ❑ Carries optional information only for the destination node(s).
- ❑ If appears before a Routing Header → processed by each listed intermediate node.
- ❑ If appears before upper-layer header → processed only by final destination.
- ❑ Format: Same as Hop-by-Hop Options Header.

Extension Header: Authentication

- ❑ This header contains a **checksum** that enables the destination to authenticate the sender.
- ❑ Provides connectionless integrity, data origin authentication, and optional anti-replay.



IPv6: Extension Header: Encapsulation



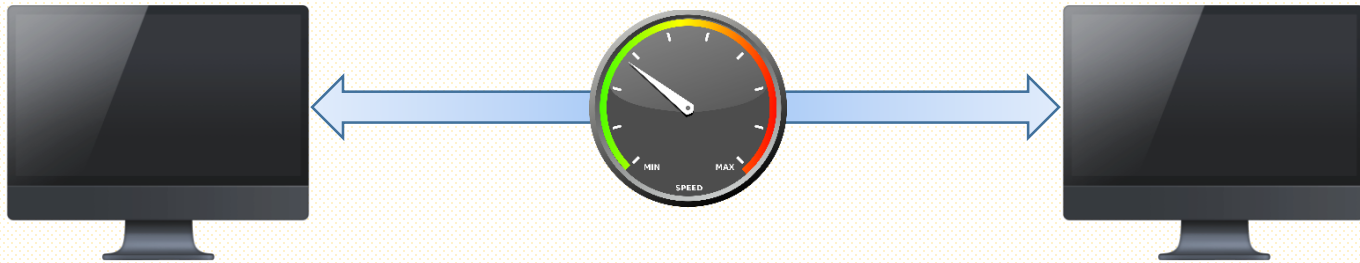
- ❑ This header contains the encryption key number and the encrypted payload.
- ❑ The destination can recover the original payload from these two items using a decryption algorithm that is left to the choice of the user.
- ❑ Provides confidentiality, integrity, authentication for payload

SPI	Sequence Number	Payload Data	Padding	Pad Len	Next Header	ICV
-----	-----------------	--------------	---------	---------	-------------	-----

Encrypted
transport or
tunnelled
packet

Integrity Check
value

Flow Control



- ❑ A mechanism that ensures the sender does not transmit more data than the receiver can process.
- ❑ Feedback based flow control:
 - Receiver ask to send more data.
 - Used in Data-Link and Transport layer.
- ❑ Rate based flow control:
 - Built-in mechanism to limit data-rate (without using feedback)
 - Mostly used in transport layer

Why Flow Control?

❑ Problems without flow control:

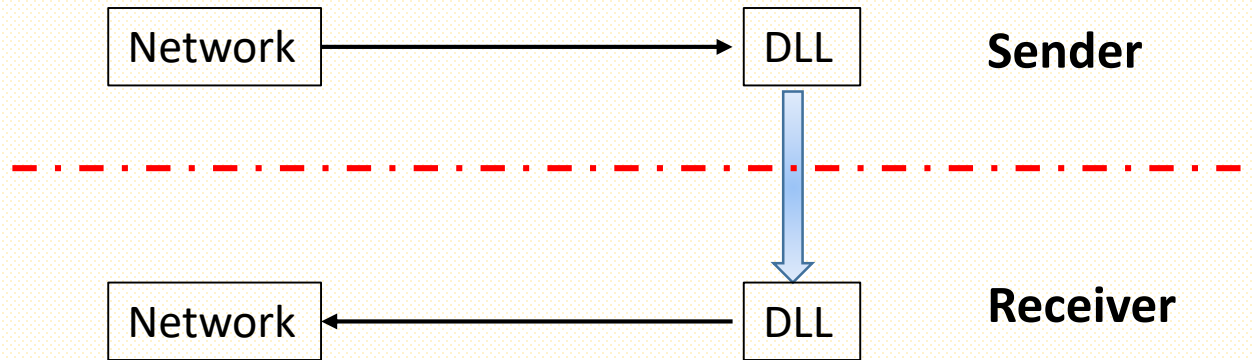
- Receiver Buffer Overflow → packet loss.
- Wasted Bandwidth → Dropped packets require retransmission.
- Data Corruption: Out-of-order or partial packets processed incorrectly.

❑ Example:

- Imagine pouring water into a glass → too fast: spill (data loss).
- Talking to someone who writes slowly → if you dictate too fast, they miss words.

❑ Example:

- Sender: 100 Mbps, Receiver: 50Mbps
- 50% data will overflow → retransmission → bandwidth waste.



❑ DLL receives frame:

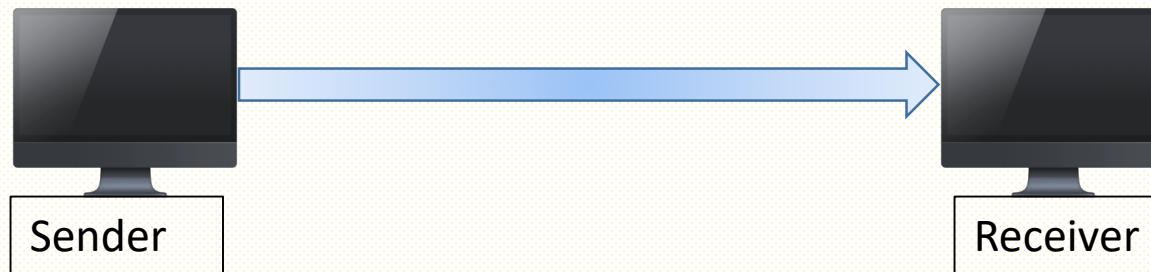
- Check header and pass to n/w layer (if no error).

❑ In general if a DLL transmits any frame it starts a timer (if not receive ACK within time; retransmit).

Communication Modes: Simplex

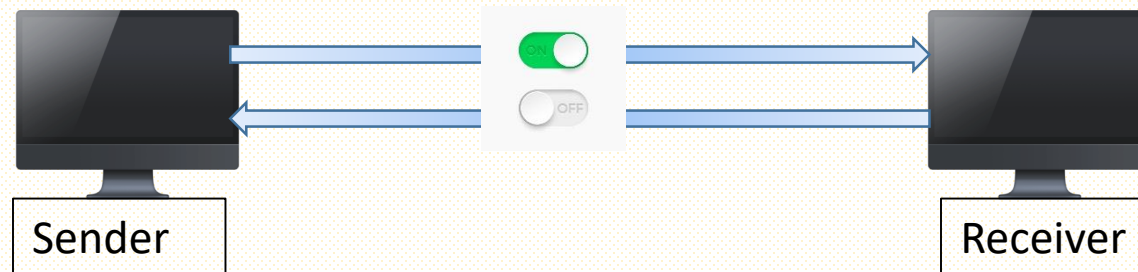


- ❑ Data flow in one direction only.
- ❑ Examples:
 - Keyboard → CPU
 - CPU → Monitor
 - PC → Printer
- ❑ Characteristics:
 - Receiver can't send ACKs unless another channel exists.



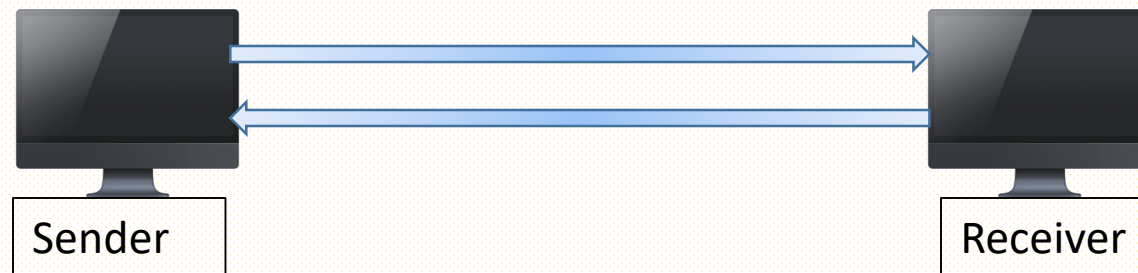
Communication Mode: Half Duplex

- ❑ Data flows in both directions, but only one direction at a time.
- ❑ Example:
 - Walkie-talkies
- ❑ Only one side transmits → need turn-taking rules + buffer management.



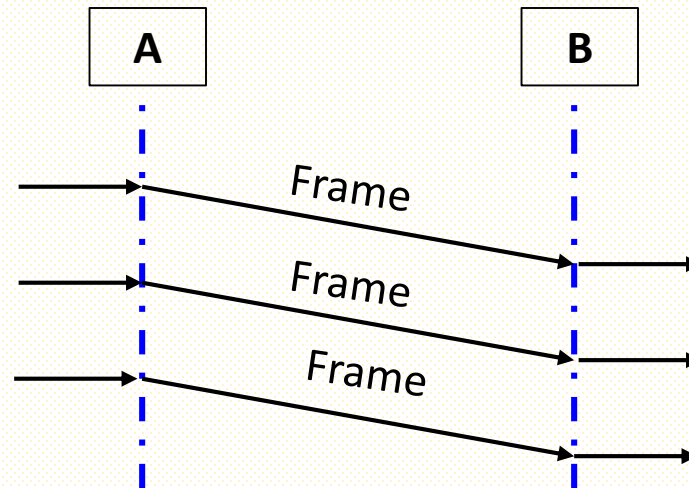
Communication Mode: Full Duplex

- ❑ Both sender and receiver can transmit data simultaneously
- ❑ Examples:
 - Telephone Call



Transmission Protocols : Utopia

- ❑ **Unrestricted Simplex Protocol**: **hypothetical** protocol designed for unidirectional data transmission over an **ideal channel**.
- ❑ **Assumption**: Sender/Receiver are **always ready for data processing + infinite buffer**.
- ❑ Unrealistic Model → **No error/flow control**



Simplex: Stop-and-Wait

❑ Utopia + flow control

❑ **Assumption:**

- Unidirectional transmission
- Error free channel
- Receiver can process data at finite rate.

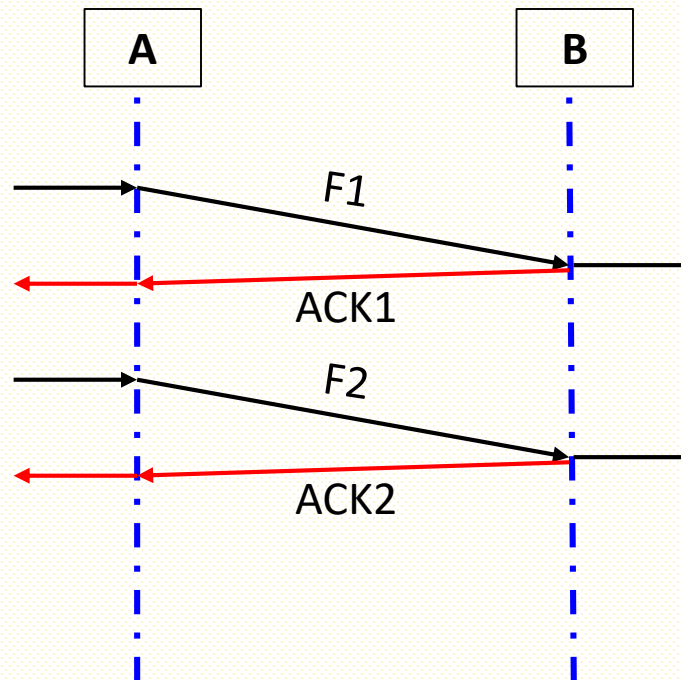
UEF

- ❑ General solution to prevent the sender from flooding receiver?
- Send feedback to sender (dummy packet: permission to transmit next packet).

Simplex: Stop-and-Wait

❑ Process:

- Sender sends one frame.
- Waits for ACK before sending the next.
- If ACK not received → retransmit.



Simplex: Stop-and-Wait

- ❑ As we are assuming simplex mode: packet will be sent in strict alternation (half duplex channel).
- ❑ An incoming packet will always be an acknowledgement.
- ❑ Advantages:
 - Simple and reliable, avoid overflow.
 - Easy duplicate detection.
- ❑ Disadvantages:
 - Inefficient for long-distance, high-speed links.
 - No error control.

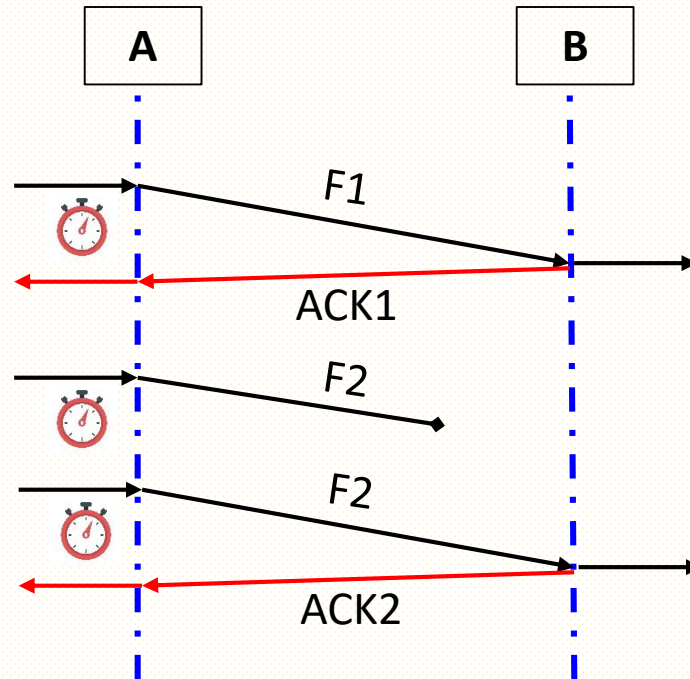
OD

Simplex: for Noisy Channel

- ❑ Simplex stop and wait with Error Control + ARQ
- ❑ Assumption:
 - Unidirectional transmission
 - Error is possible in channel (Error can be in Data or in ACK)
 - Receiver can process data at finite rate.
- ❑ Solution
 - What if Data frame lost?
 - But what if ACK lost?

Simplex: for Noisy Channel

- ❑ Solution Approach 1: Use a timer at sender end.



- ❑ If ACK not received before timer ends → retransmission.
- ❑ Receiver will transmit ACK only if the received data is correct.

Simplex: for Noisy Channel



❑ Problem with Approach 1?

- What if ACK get damage.
- How long the sender should wait before retransmission.
- If ACK lost → Duplicate packets.

Simplex: for Noisy Channel

- ❑ Solution: Need to check if receiver is seen any packet for the first time.
 - It can be handled using sequence number (assign sequence number to each data).
- ❑ How many bits should be added in header file as sequence number?
 - If frame m lost, the packet $m+1$ will not be sent.
 - If ACK of m is lost, m will be tried again and again and $m+1$ will be in waiting.
 - Transmission of $m+1$ is possible if sender receives ACK of m .
 - ACK of m indicates $m-1$ is already delivered.
- ❑ Thus 1-bit sequence number is sufficient.

Simplex: for Noisy Channel

❑ Approach 2: Sequence Number + Timer.

- When a packet with correct sequence# arrives → process and forward and send ACK.
- Increase the sequence# (to expected sequence#)
- Any packet with wrong sequence# will be rejected.

Sender

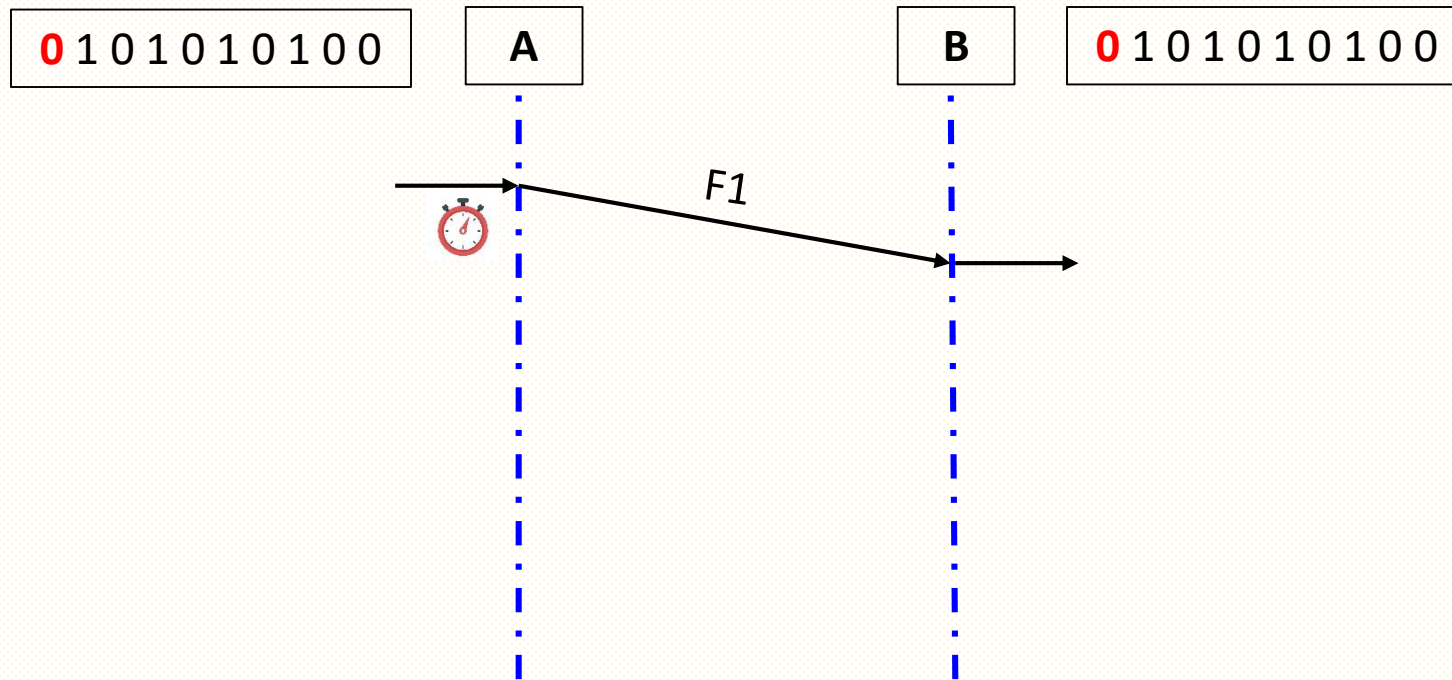
0 1 0 1 0 1 **0** 1 0 1 0 1 0 1 0

Receiver

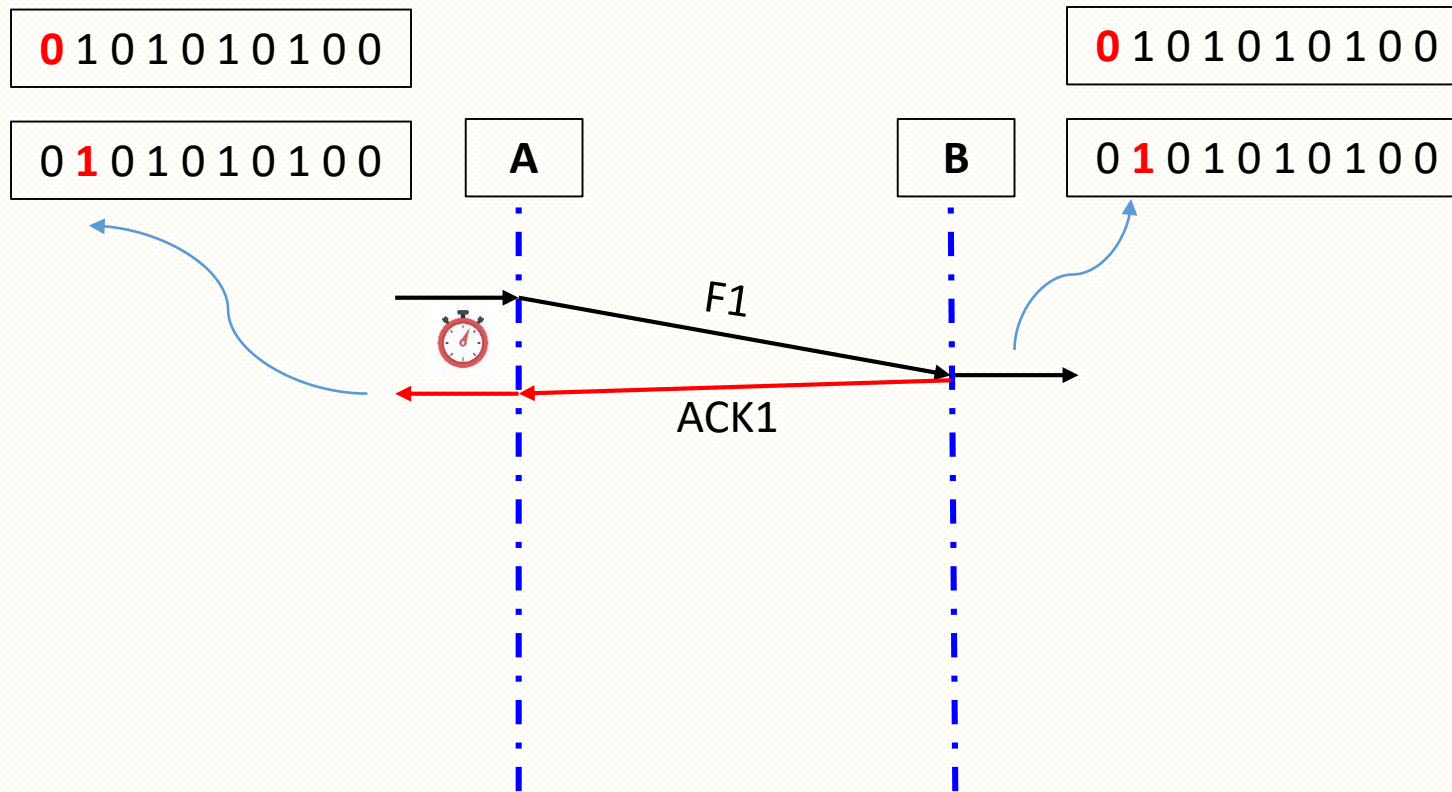
0 1 0 1 0 1 **0** 1 0 1 0 1 0 1 0

- ## ❑ This type of protocol where sender wait for positive ACK before sending next data is known as ARQ (automatic repeat request) or PAR (Positive ACK with Retransmission).

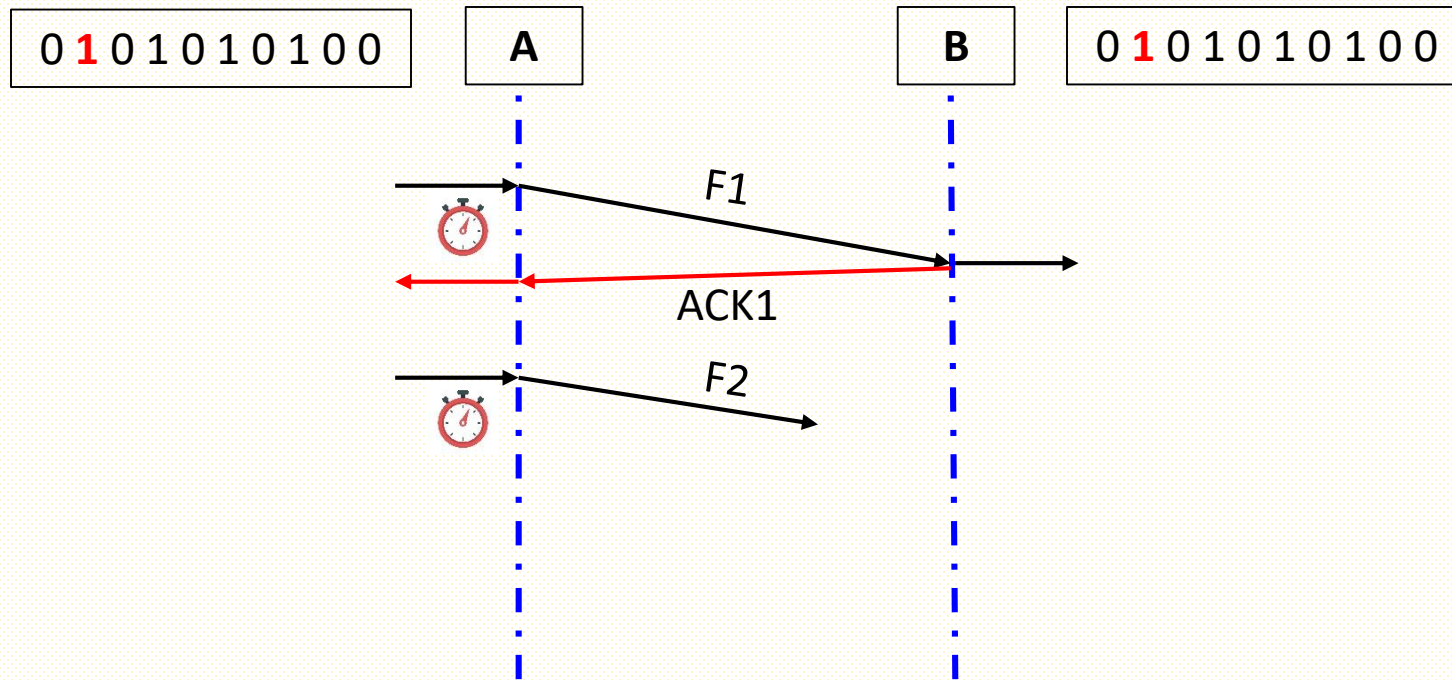
Simplex: for Noisy Channel



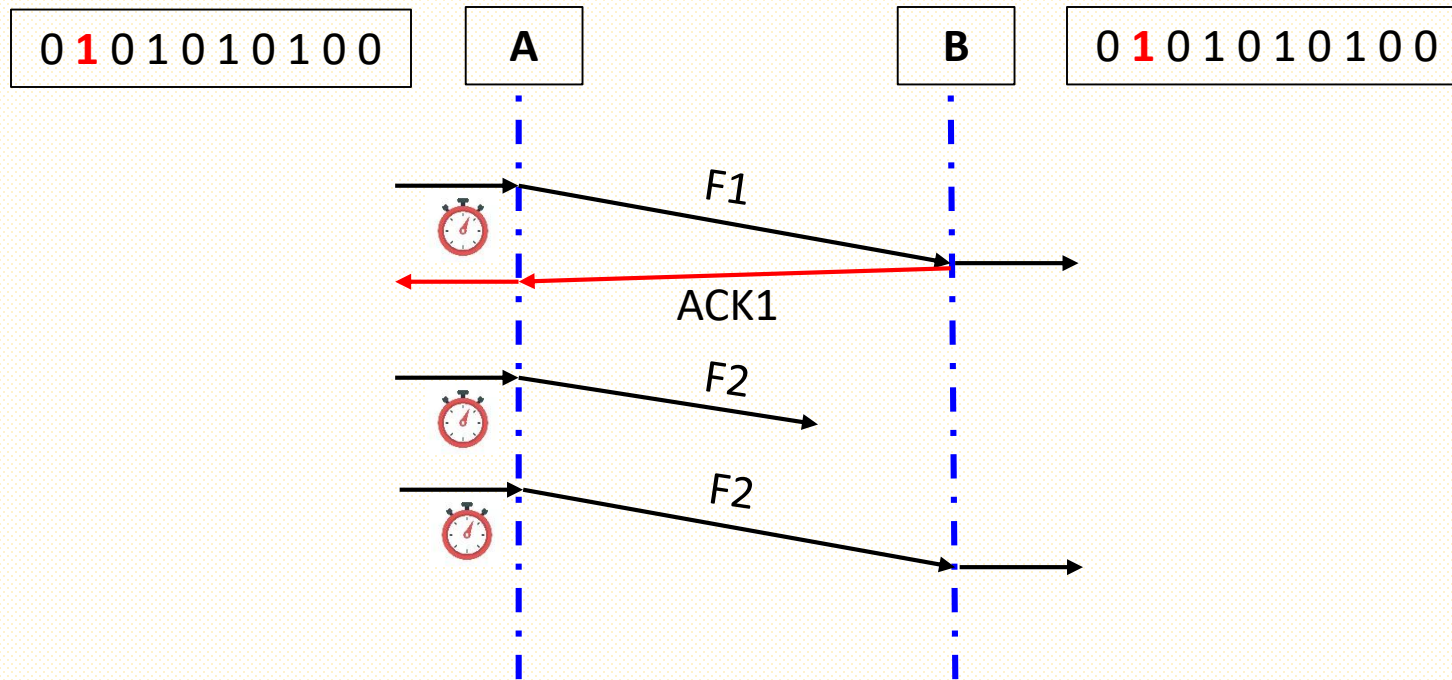
Simplex: for Noisy Channel



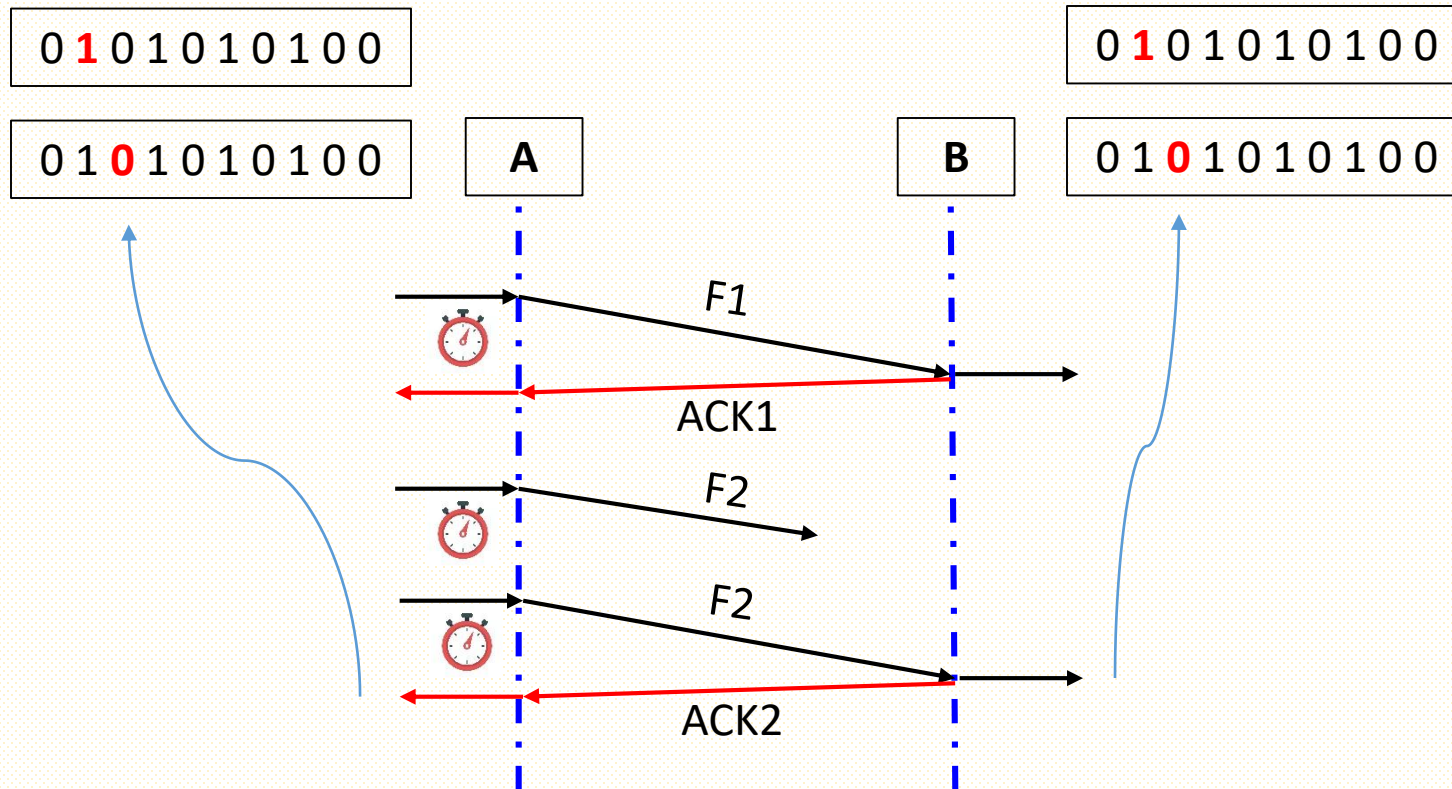
Simplex: for Noisy Channel



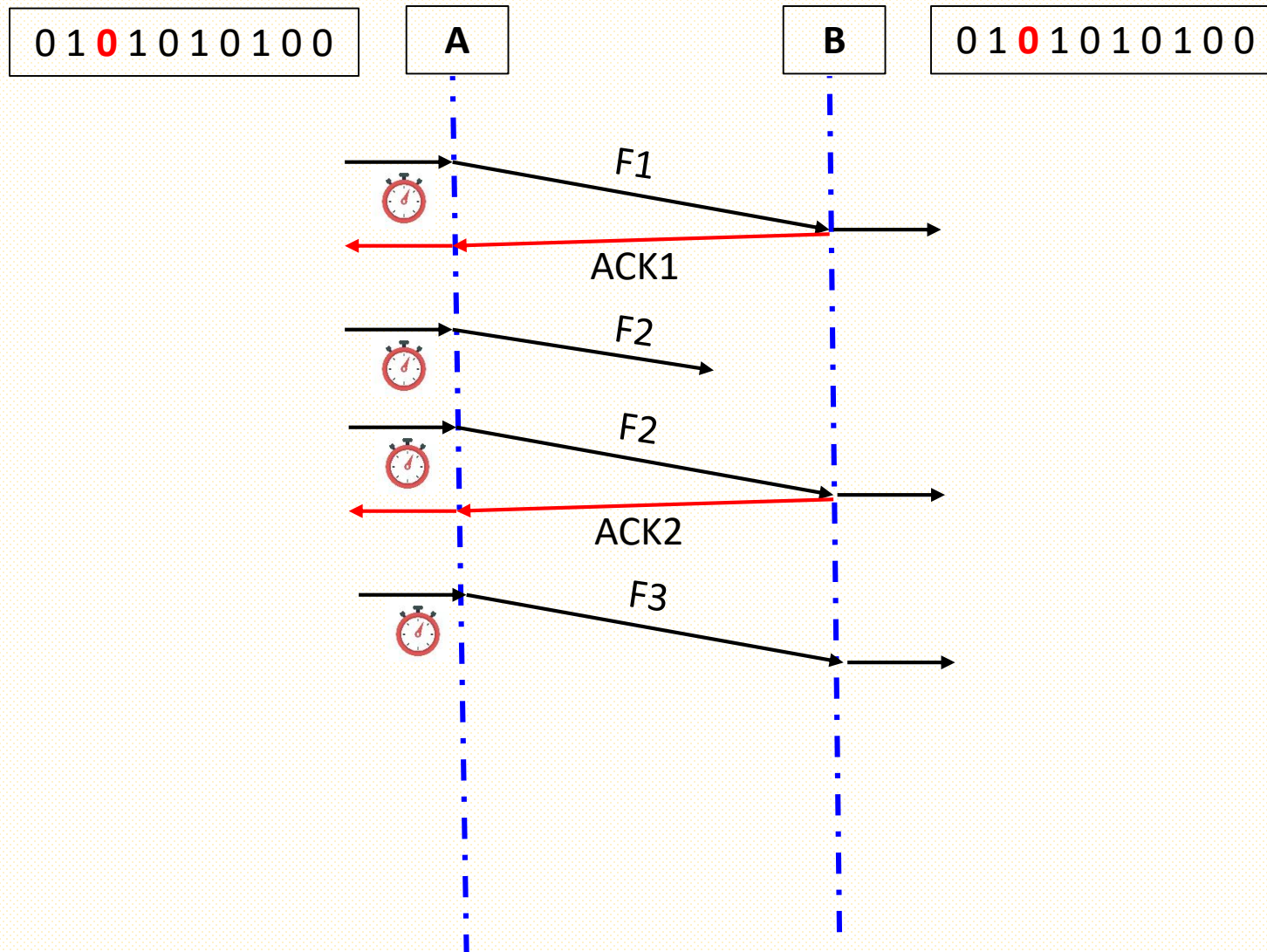
Simplex: for Noisy Channel



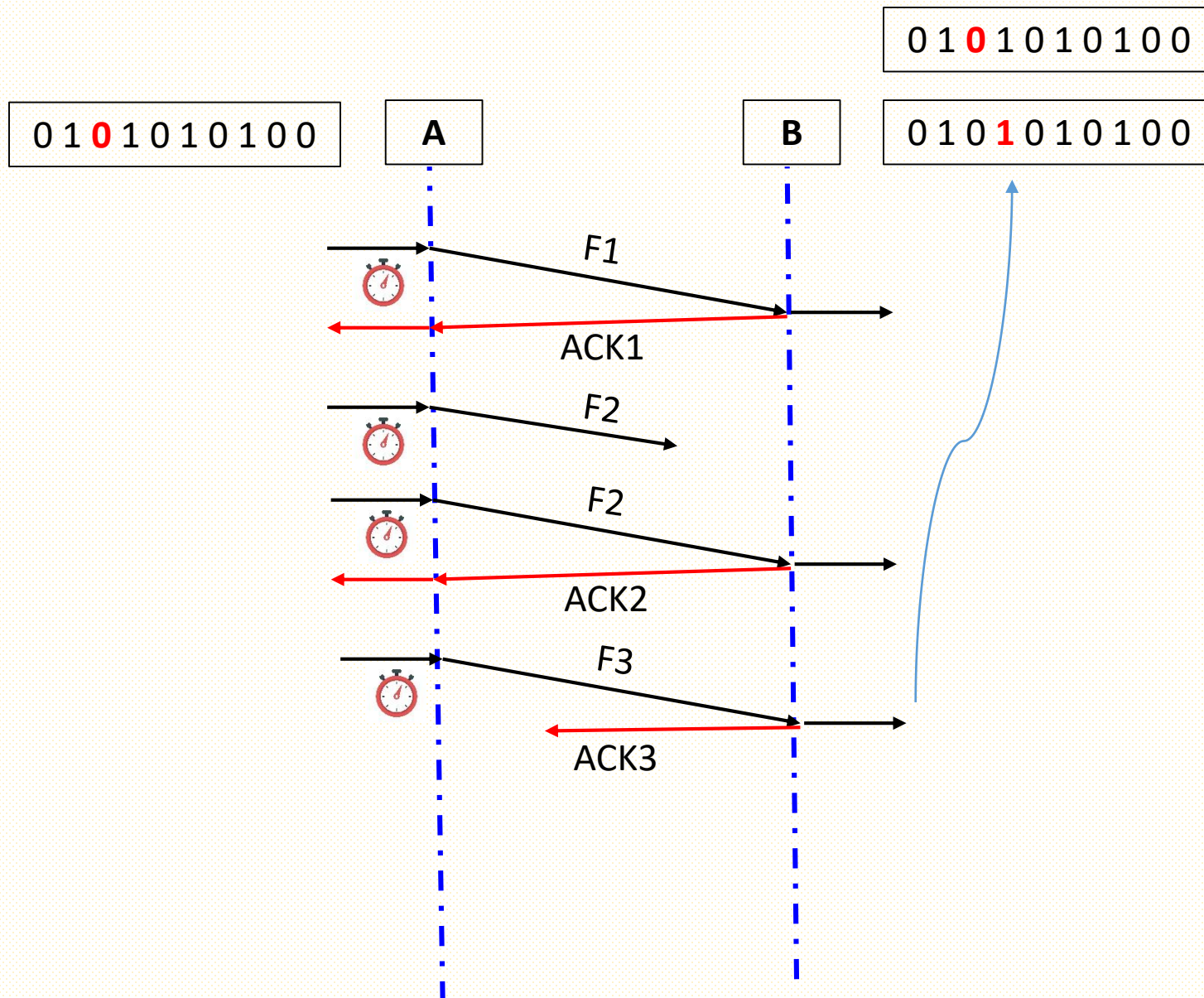
Simplex: for Noisy Channel



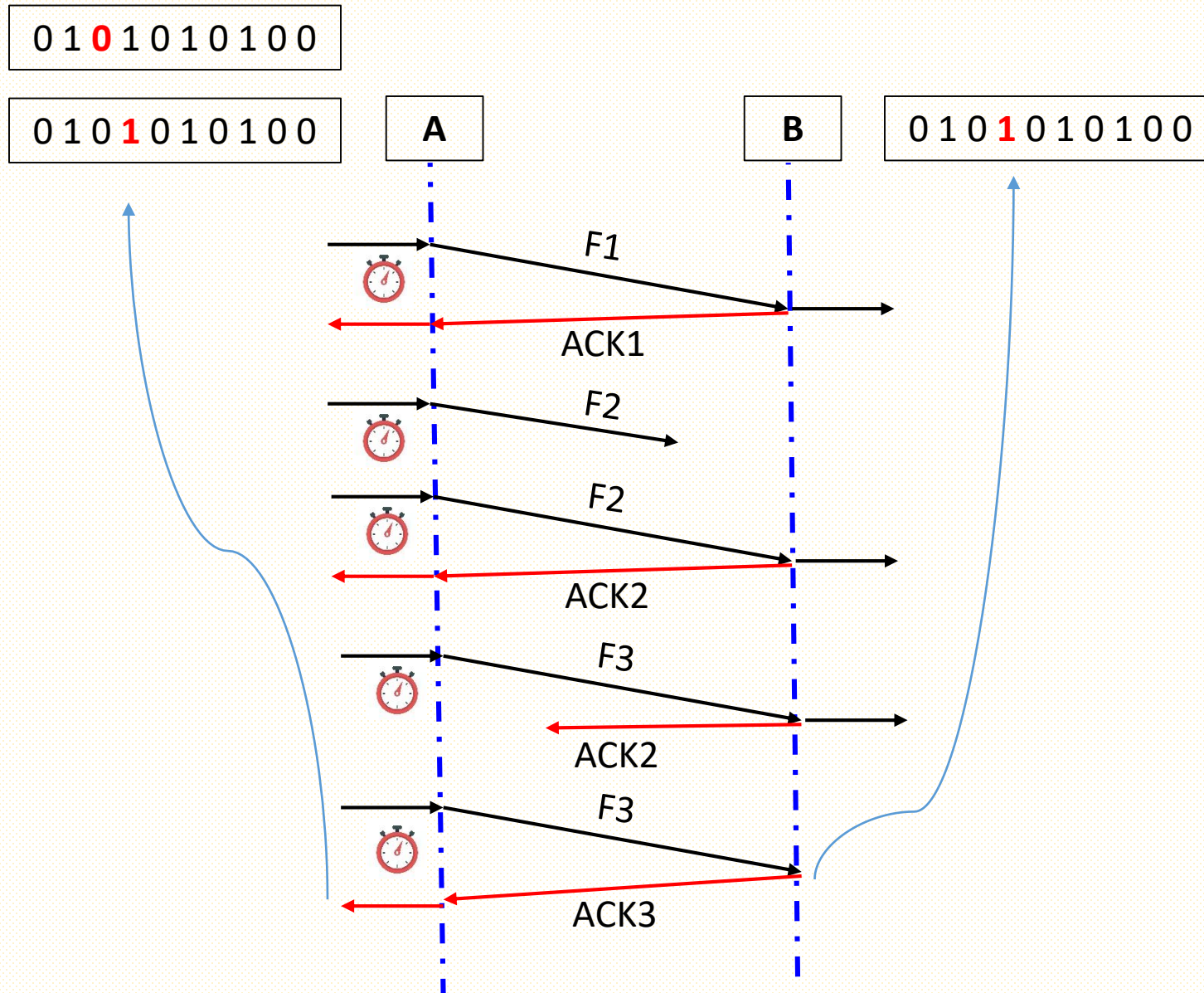
Simplex: for Noisy Channel



Simplex: for Noisy Channel

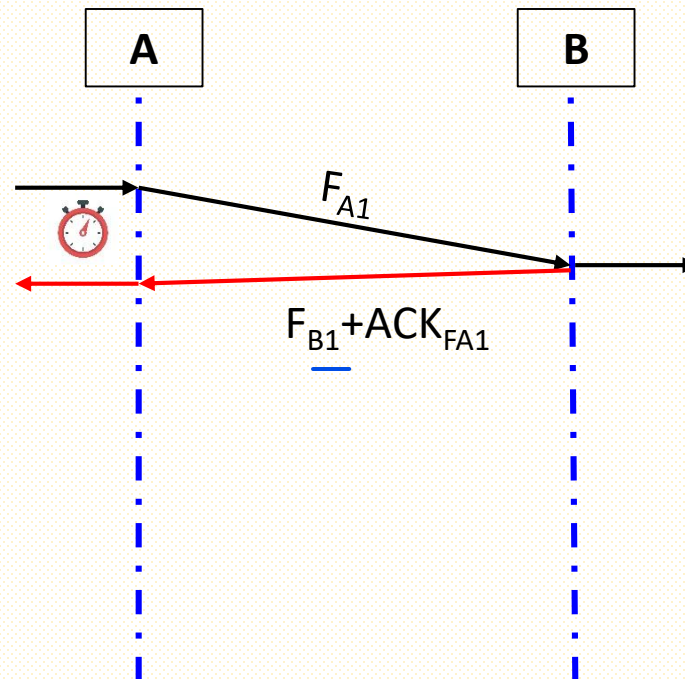


Simplex: for Noisy Channel



Piggybacking

- ❑ Used for bidirectional transmission.
- ❑ Piggybacking is a technique of delaying outgoing ACK and attaching it to the next data packet.
 - If a data frame received instead of immediately sending ACK, wait for network layer if it have any frame to be transmitted.



❑ Transmission Delay: time to transmit packet from host to outgoing link.

- $T(t) = \frac{D}{B}$
- $D \rightarrow$ Frame size.
- $B \rightarrow$ Bandwidth.

❑ Propagation Delay: outgoing link to receiver.

- $T(p) = \frac{d}{s}$
- $d \rightarrow$ distance.
- $s \rightarrow$ wave propagation speed.

❑ Efficiency:

❑ $\eta = \frac{T(t)}{T(t) + 2T(p)}$

Numerical: Example 1

- ❑ Frame size 10KB, Propagation delay = 50 ms, Link Speed 1Mbps:
- ❑ Transmission Time $T(t)$?
 - 81.92ms
- ❑ Efficiency η ?
 - 45%

$$10 \times 10^3 \times 10^6$$

Numerical: Example 2



- ❑ Frame size 5000 bit, Propagation delay = 20 ms, Link Speed 100 kbps:
- ❑ Transmission Time $T(t)$?
 - 0.02 s
- ❑ Efficiency η ?
 - 55.5%

Numerical: Example 3

- ❑ 20kbps satellite link, propagation delay 400 ms, frame size 100 byte, max data rate possible?
Handwritten: 20,000
- ❑ Transmission Time $T(t)$?
 - 40ms
- ❑ Efficiency η ?
 - 4.76%
- ❑ Max Data Rate (throughput)?
 - $\eta * \text{link speed} \rightarrow 952 \text{ bps}$

Numerical: Example 3



- ❑ Earth $\rightarrow$ planet (9×10^{10} meter) away, frame size 32KB, link speed 64Mbps, channel utilization? (propagation speed = speed of light = 3×10^8 m/s)
- ❑ Transmission Time $T(t)$?
 - 4.096ms
- ❑ Propagation Time $T(p)$?
 - 300 s
- ❑ Efficiency (channel utilization)?
 - 0.0006827%
- ❑ Utilization?
 - 437 bps

Numerical: Example 4



❑ In stop and wait protocol, every 4th packet is lost, we need to send 10 packets, how many transmission required to send all the packets?

❑ 13

Increase Efficiency



- ☐ Increase frame size. ^
- ☐ Reduce propagation delay. ✓
- ☐ Use pipelining protocols (e.g., Sliding Window).

Negative Acknowledgment



❑ Definition:

- Receiver indicates an error occurred in the received frame.
- Purpose: Faster recovery since sender doesn't wait for timeout.
- Example: Email server says “Delivery failed”.

Timeout Value?

- ❑ Definition: A timer in sender that triggers retransmission if ACK/NAK not received in time.
- ❑ If RTT = 200 ms (2xTp)
- ❑ processing delay → 10 ms,
- ❑ What should be value of timeout?
 - $T_{timeout} > 200 + 10 = 210 \text{ ms.}$

Routing Algorithm

- ❑ Routing → Process of selecting best path for data packets.
- ❑ Routers use routing tables to forward packets.
- ❑ Two main types:
 - Static Routing
 - Dynamic Routing
- ❑ Main categories:
 - Distance Vector Routing (RIP, BGP)
 - Link-State Routing (OSPF)

Static Routing

- ❑ Routes are manually configured by network administrator (no automatic update)
- ❑ `ip route [dest_net] [sub_mask] [next_hop]`
- ❑ Advantage
 - Simple to configure (for small network)
 - No CPU or RAM overhead (low resource usage)
 - Full control by admin (no unwanted route)
- ❑ Disadvantage
 - Not scalable
 - Require manual update if topology changes.
 - Prone to error if many routes.

Dynamic Routing

- ❑ Routers learn routes automatically via protocols. (automatic update). It uses routing algorithms to select best path.
- ❑ `ip route [dest_net] [sub_mask] [next_hop]`
- ❑ Protocols
 - RIP, OSPF, EIGRP, BGP
- ❑ Advantage
 - Auto route discovery → scalable for large network.
 - Support fault tolerance.
- ❑ Disadvantage
 - Consume CPU, RAM, Bandwidth
 - Complex configuration.
 - Take time to converge

ds n

Best Path Selection

- ❑ Best path → shortest path
- ❑ What is shortest path: minimum cost b/w source and destination
 - Hop count
 - Bandwidth use
 - Delay/latency
 - Link reliability

H B L L

❑ Distance Vector:

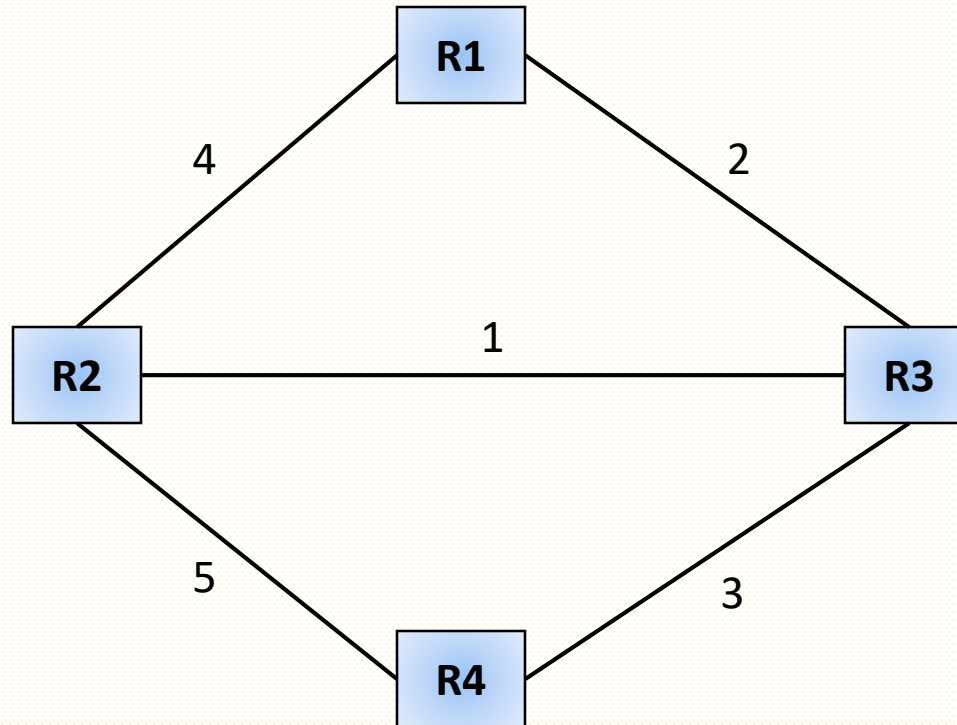
- Each router knows the complete network topology. Flood data to all routers. Each router then build a network graph.
- Key Idea: ***Tell me what you know, I will update mine***
- Algorithm used → Bellman-Ford.
- Take time to converge.

❑ Link State:

- Each router maintains a vector (table) of distances to all destinations. Routers exchange tables periodically with neighbors.
- Key Idea: ***I learn the whole map; then will compute shortest path.***
- Algorithm used → Dijkstra.

Bellman-Ford Algorithm

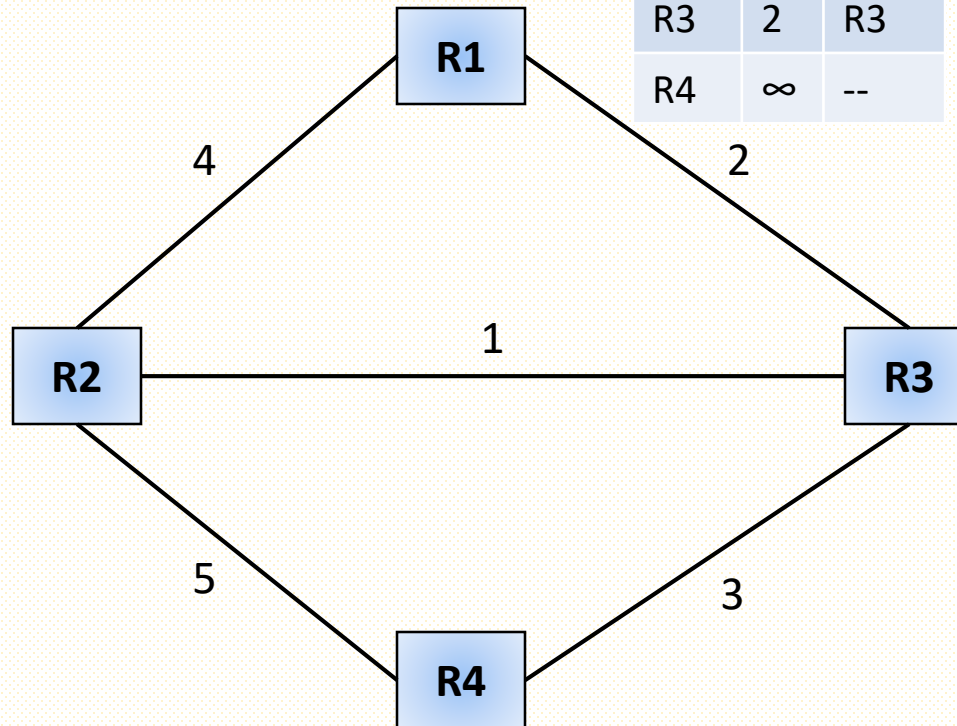
- ❑ Shortest path from single source to all other vertices.
- ❑ Weight = link cost



Bellman-Ford Algorithm

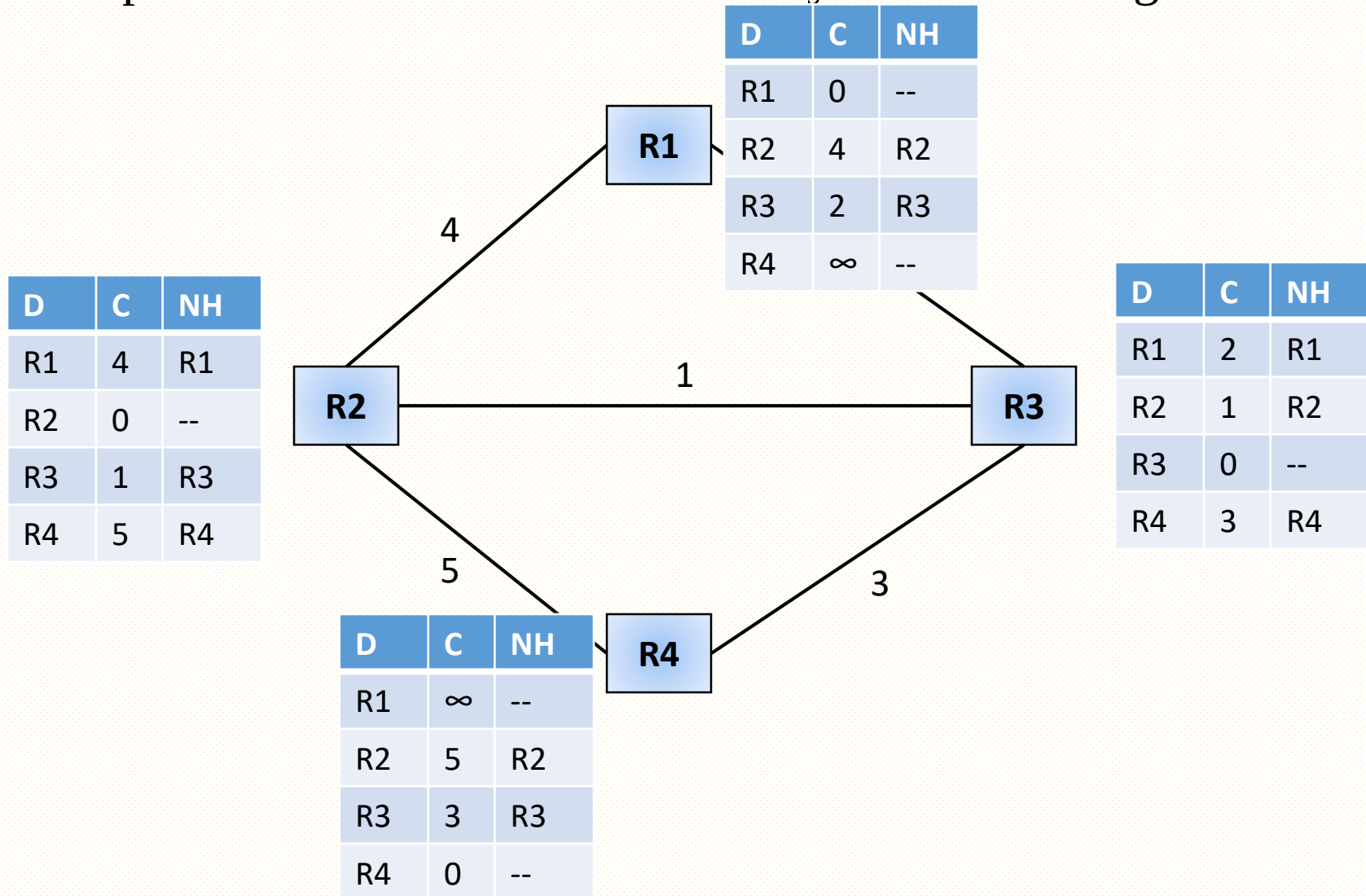
- Step1: each router knows the directly connected neighbors.

D	C	NH
R1	0	--
R2	4	R2
R3	2	R3
R4	∞	--



Bellman-Ford Algorithm

- Step1: each router knows the directly connected neighbors.



Bellman-Ford Algorithm

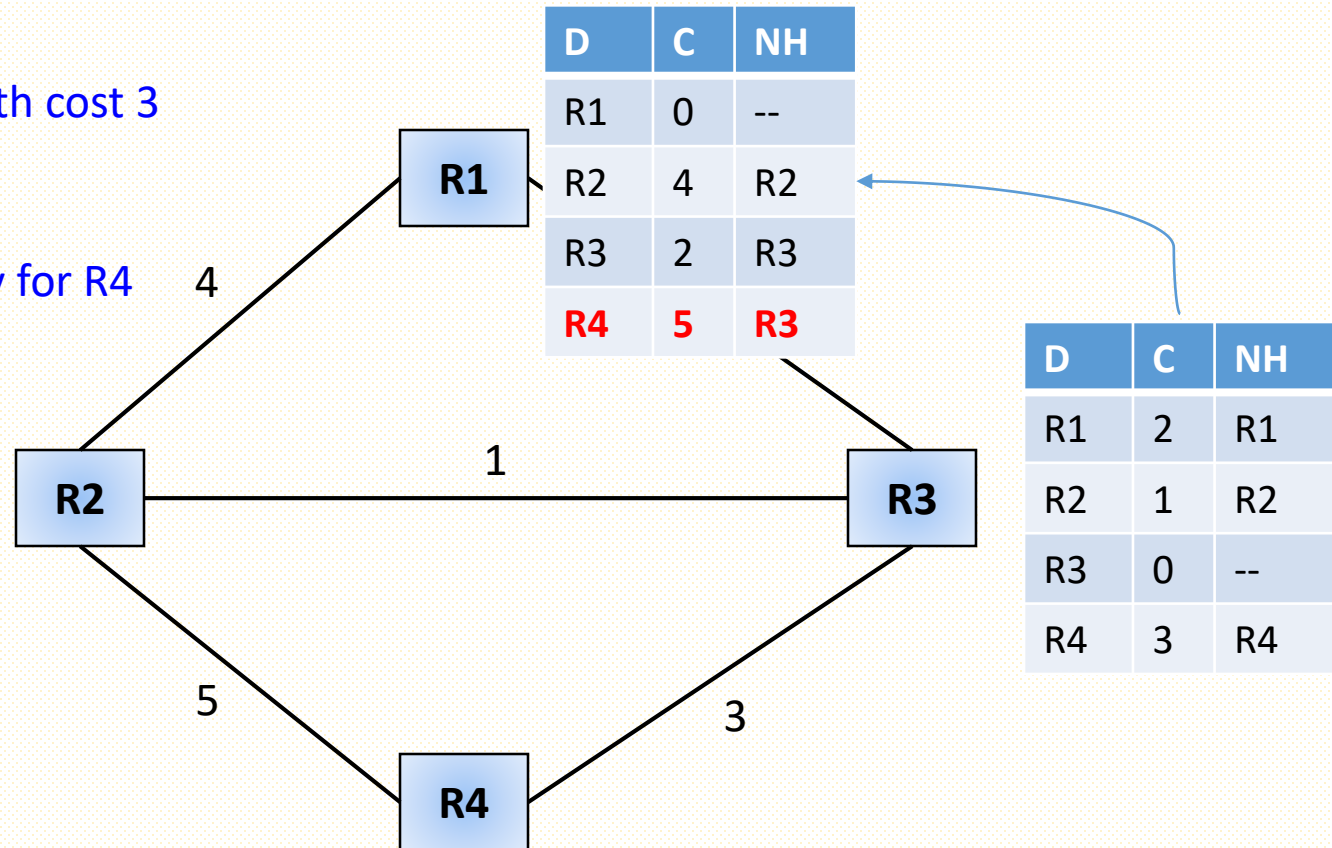
- Step 2: Routers exchange their distance vectors with neighbors and update. (*only the destination and distance*)

R1 hears from R3

C says: I can reach R4 with cost 3

R1 to R4 cost 2+3

$\infty > 5$? Yes, update entry for R4



Bellman-Ford Algorithm

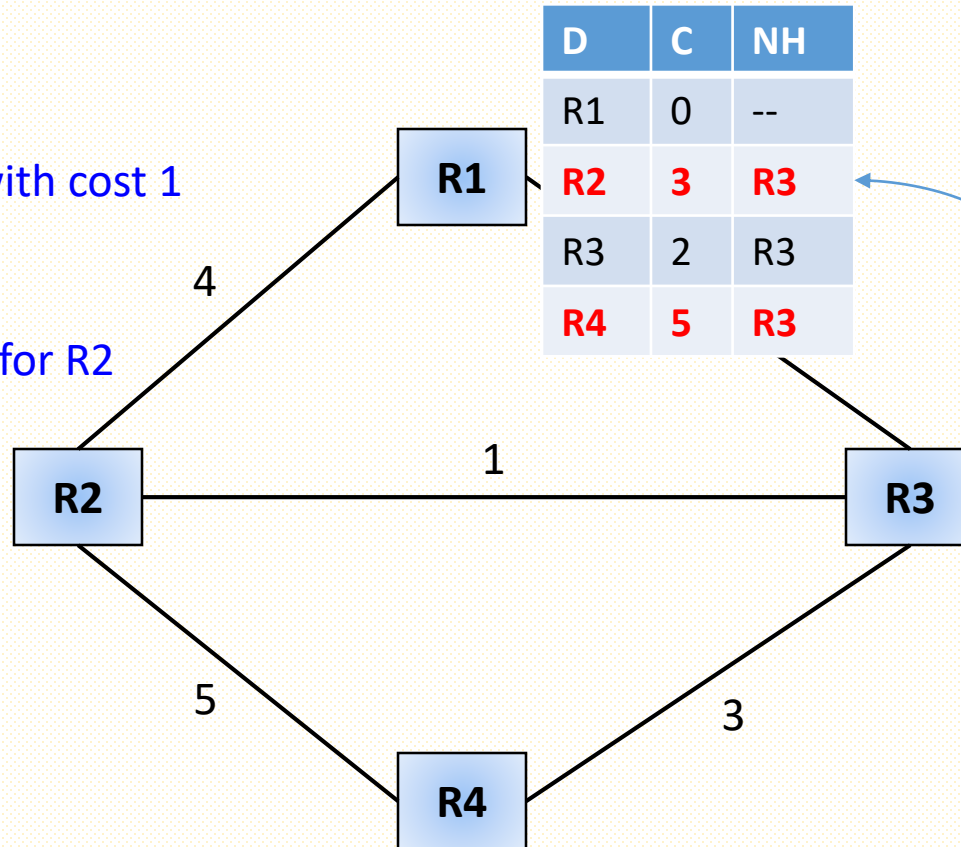
- ❑ Step 2: Routers exchange their distance vectors with neighbors and update.

R1 hears from R3

R3 says: I can reach R2 with cost 1

R1 to R2 cost 2+1

4 > 3? Yes, update entry for R2



D	C	NH
R1	0	--
R2	3	R3
R3	2	R3
R4	5	R3

D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	3	R4

Bellman-Ford Algorithm

- Step 2: Routers exchange their distance vectors with neighbors and update.

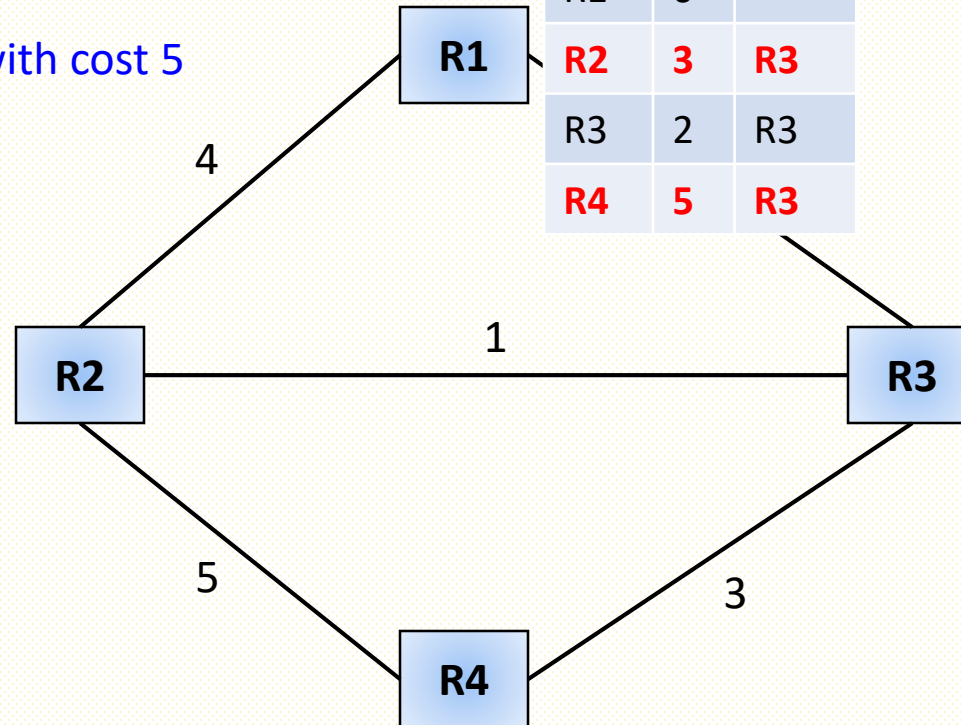
R1 hears from R2

R2 says: I can reach R4 with cost 5

R1 to R4 cost $4 + 5 = 9$

$5 > 9?$ No, No update

D	C	NH
R1	4	R1
R2	0	--
R3	1	R3
R4	5	R4



D	C	NH
R1	0	--
R2	3	R3
R3	2	R3
R4	5	R3

D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	3	R4

Bellman-Ford Algorithm

- Step 2: Routers exchange their distance vectors with neighbors and update.

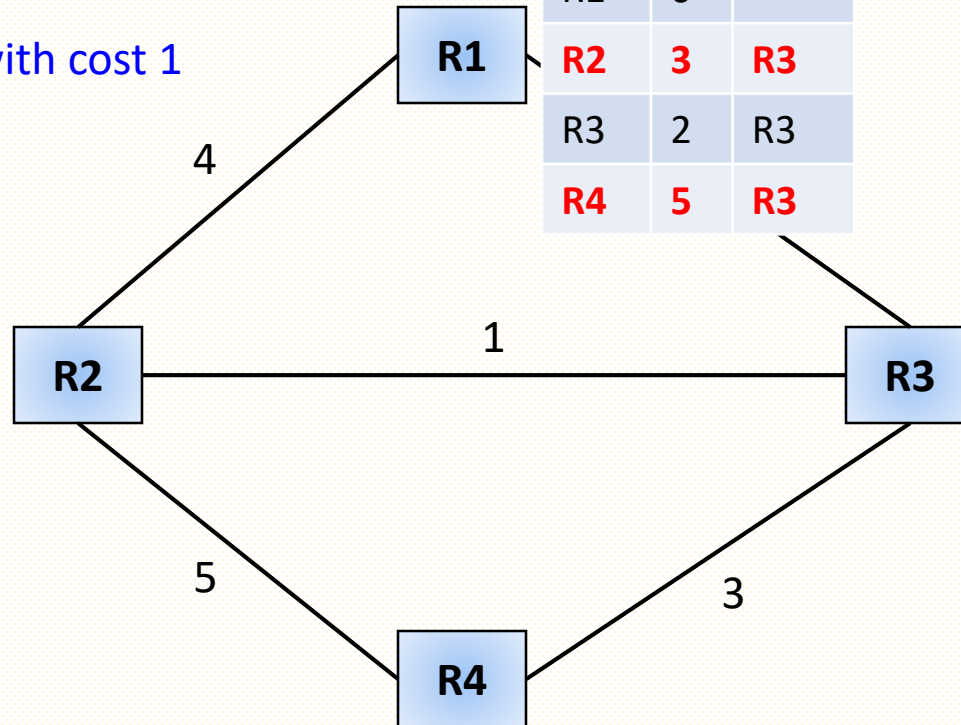
R1 hears from R2

R2 says: I can reach R3 with cost 1

R1 to R3 cost $4 + 1 = 5$

$3 > 5?$ No, No update

D	C	NH
R1	4	R1
R2	0	--
R3	1	R3
R4	5	R4



D	C	NH
R1	0	--
R2	3	R3
R3	2	R3
R4	5	R3

D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	3	R4

Bellman-Ford Algorithm

❑ Step 3: Continue exchanges.

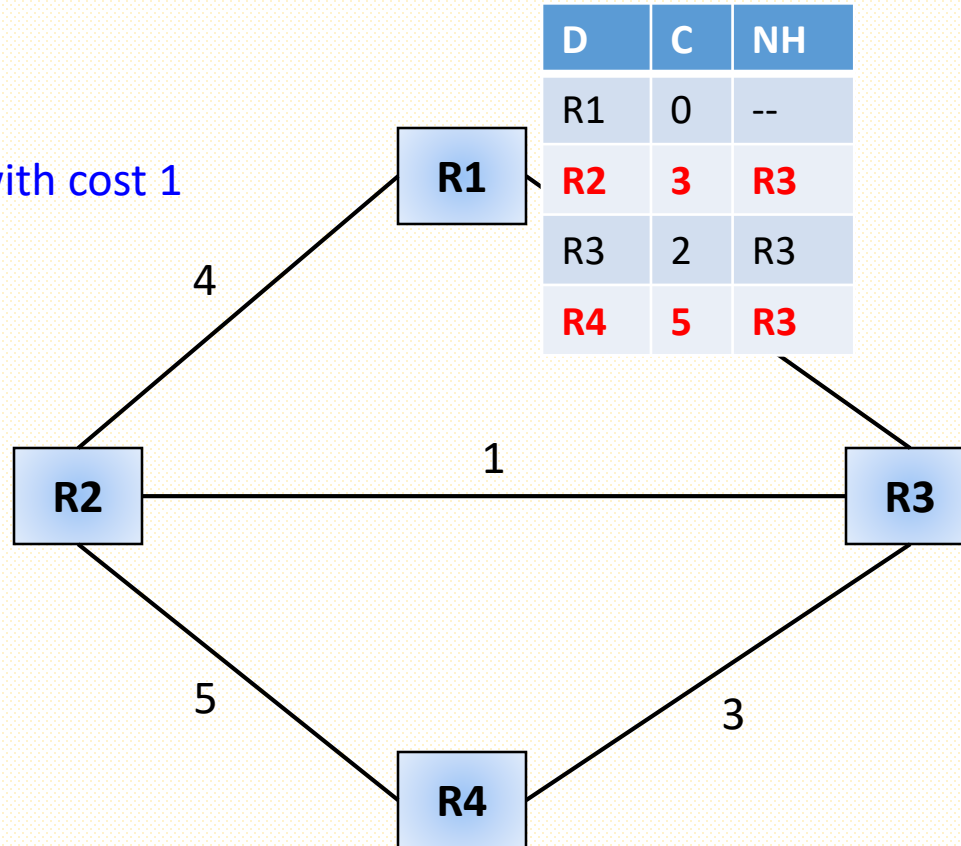
R1 hears from R2

R2 says: I can reach R3 with cost 1

R1 to R3 cost $4 + 1 = 5$

$3 > 5$?, No, No update

D	C	NH
R1	4	R1
R2	0	--
R3	1	R3
R4	5	R4

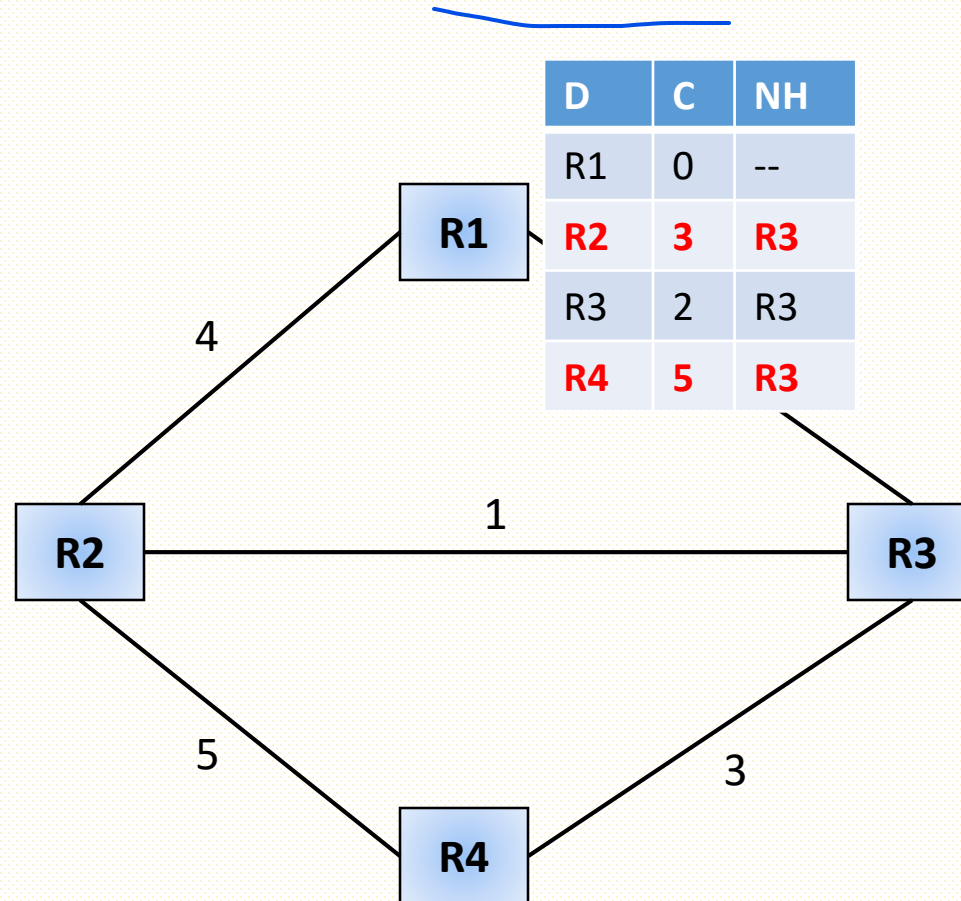


D	C	NH
R1	0	--
R2	3	R3
R3	2	R3
R4	5	R3

D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	3	R4

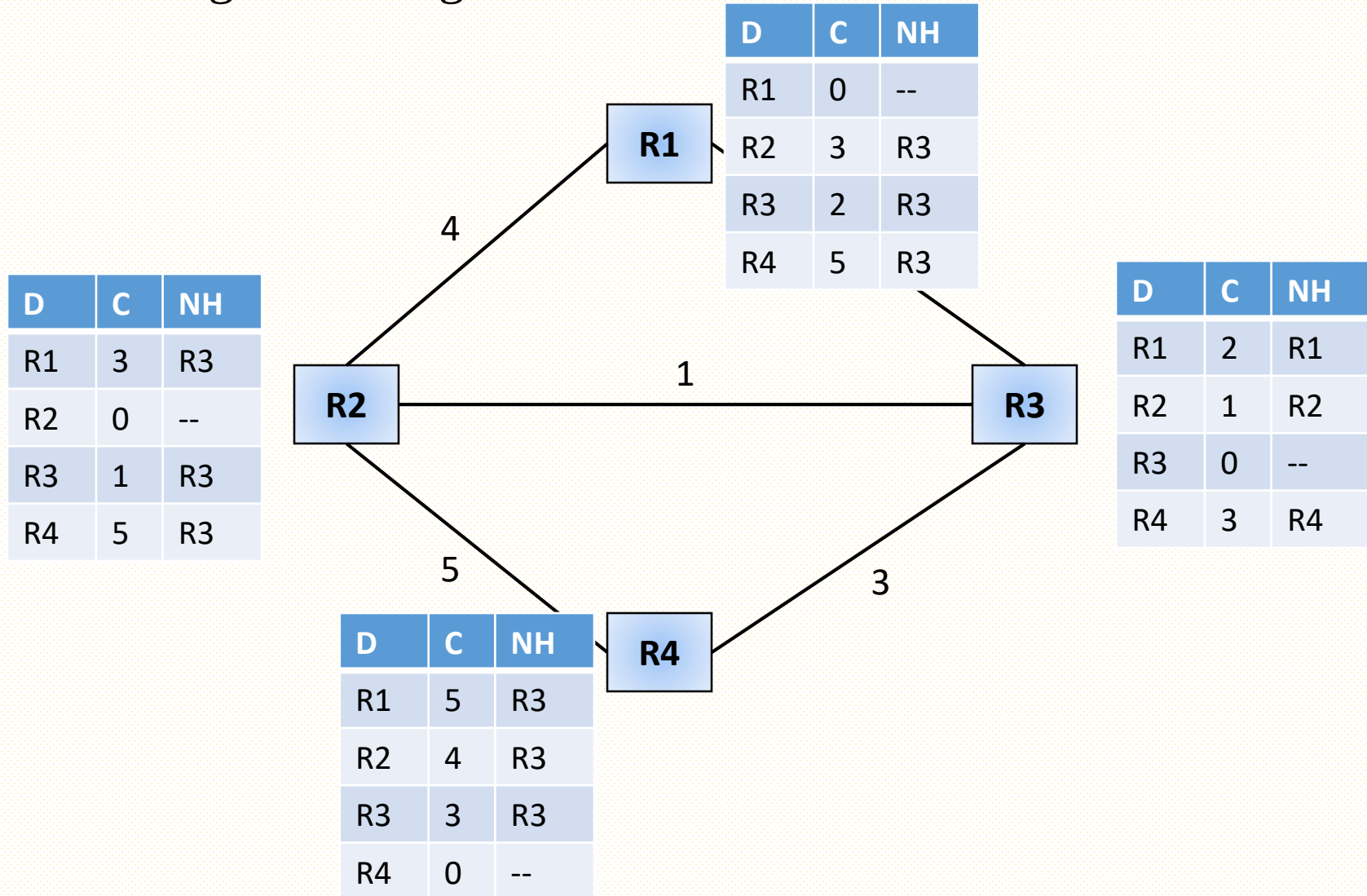
Bellman-Ford Algorithm

- Step 4: After round of exchanging vectors (until no update), all routers agree on shortest paths (V-1 iterations).



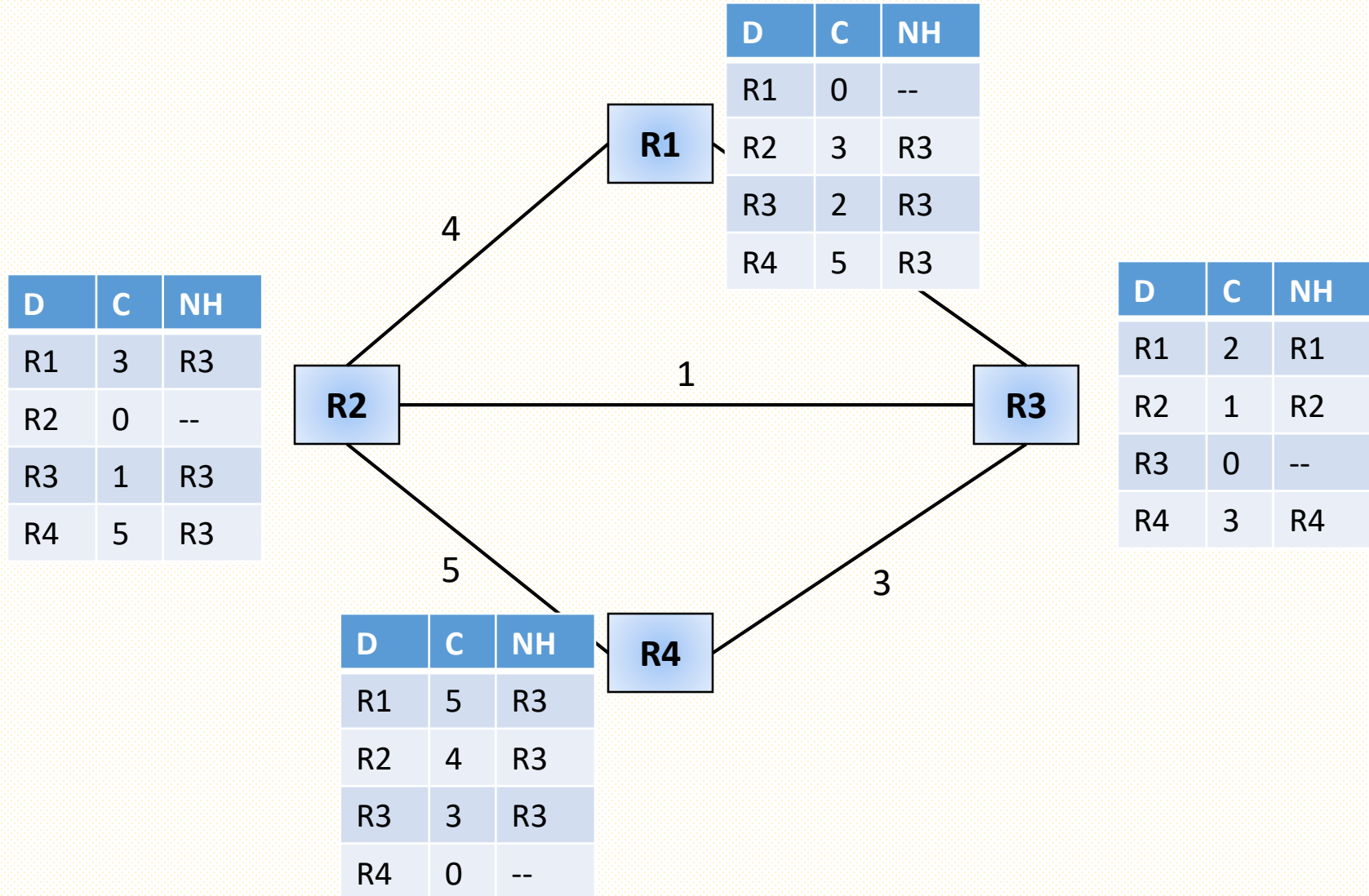
Bellman-Ford Algorithm

❑ Converged Routing Table



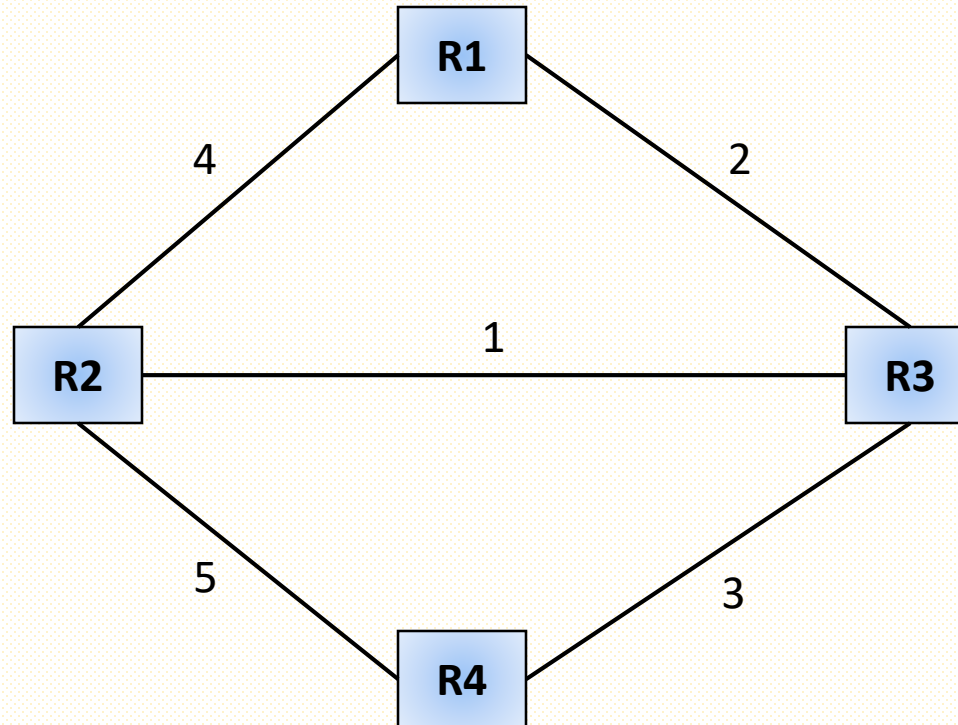
Count to ∞ Problem

❑ Assume Link R3 to R4 Fails



Count to ∞ Problem

❑ C will update its routing table.



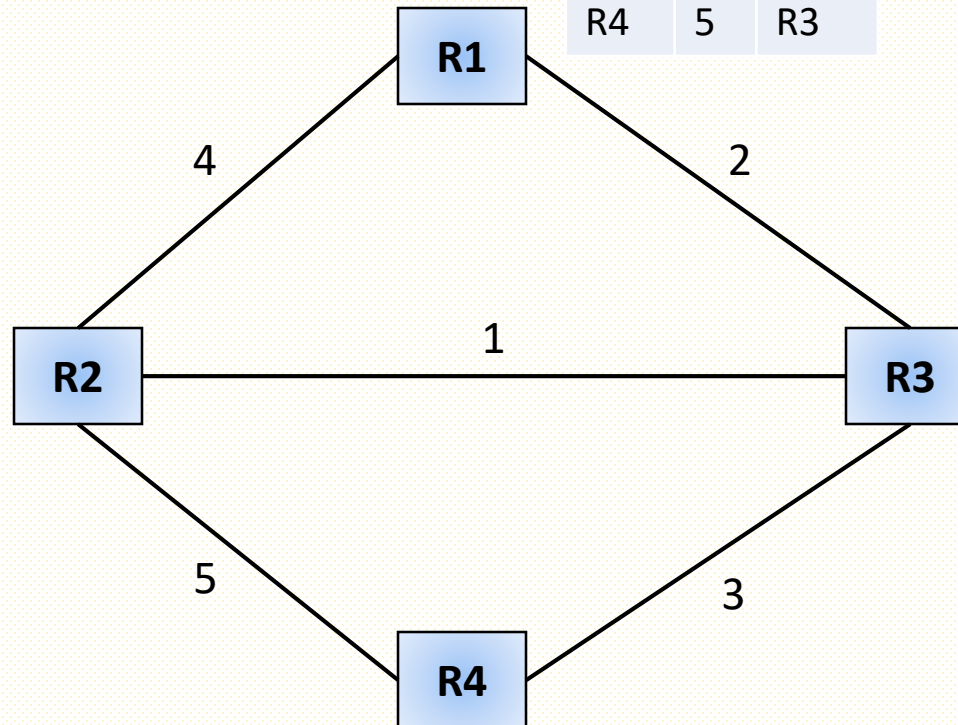
D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	∞	--

Count to ∞ Problem

- ❑ R2 think it can reach R4 by cost 4 via R3.
- ❑ It will share that info to others

D	C	NH
R1	0	--
R2	3	R3
R3	2	R3
R4	5	R3

D	C	NH
R1	3	R3
R2	0	--
R3	1	R3
R4	4	R3



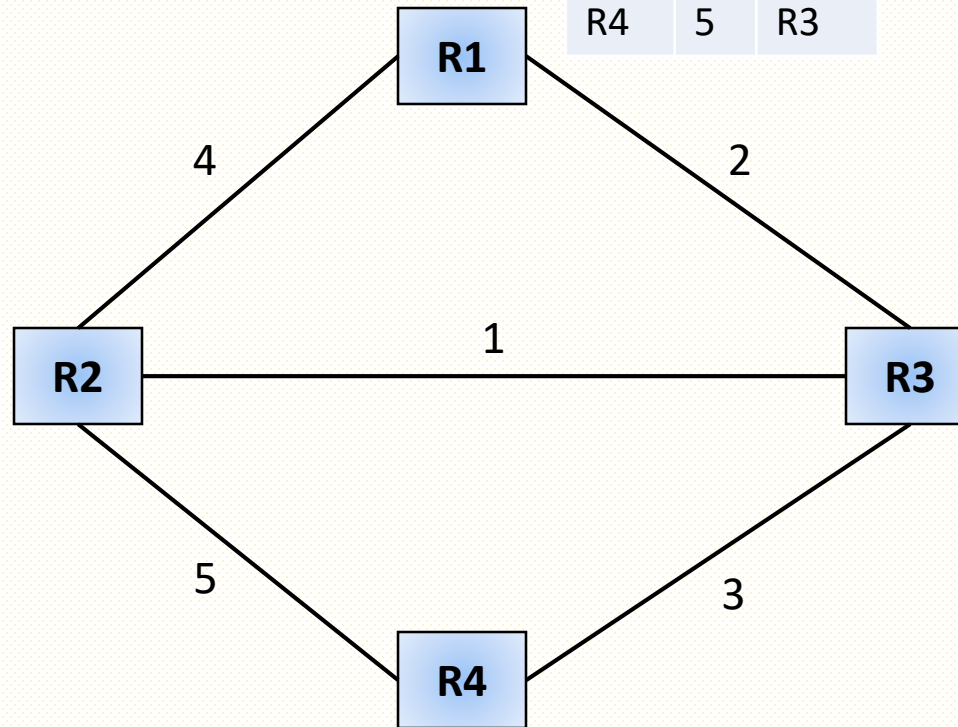
D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	∞	--

Count to ∞ Problem

- ❑ R3 get the info and thinks it can reach R4 via R2.
- ❑ **Cost $1 + 4 = 5$**
- ❑ It will share that info to others

D	C	NH
R1	0	--
R2	3	R3
R3	2	R3
R4	5	R3

D	C	NH
R1	3	R3
R2	0	--
R3	1	R3
R4	5	R3

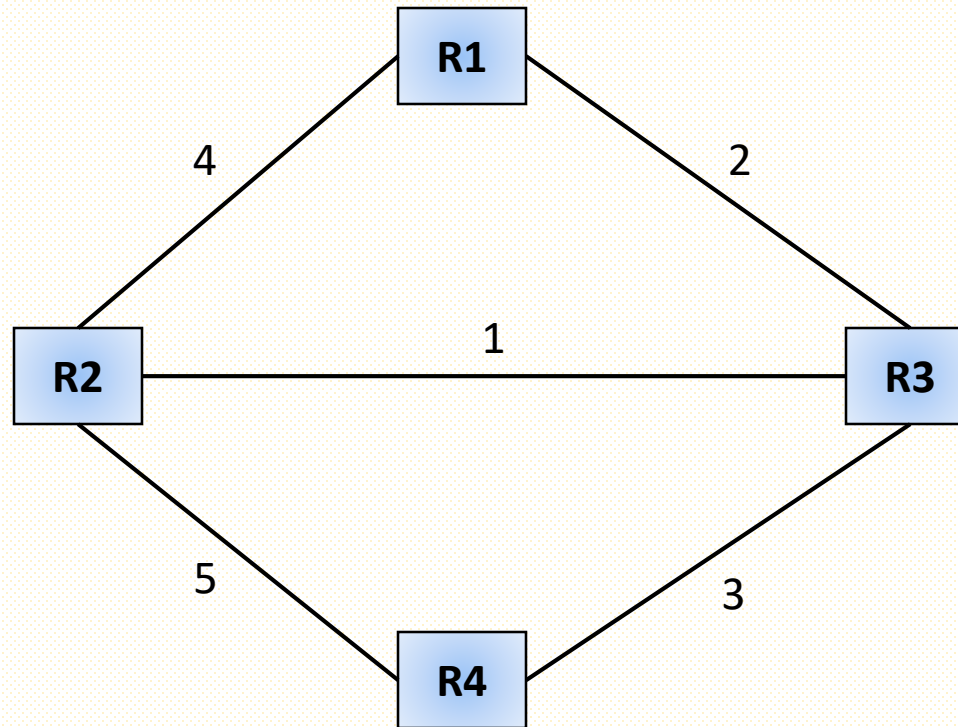


D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	5	R2

Count to ∞ Problem

- ❑ Now the loop begins
- ❑ R3 will tell R1 and R2: I can reach R4 with cost 5.
- ❑ *This will continue.*

D	C	NH
R1	3	R3
R2	0	--
R3	1	R3
R4	6	R3

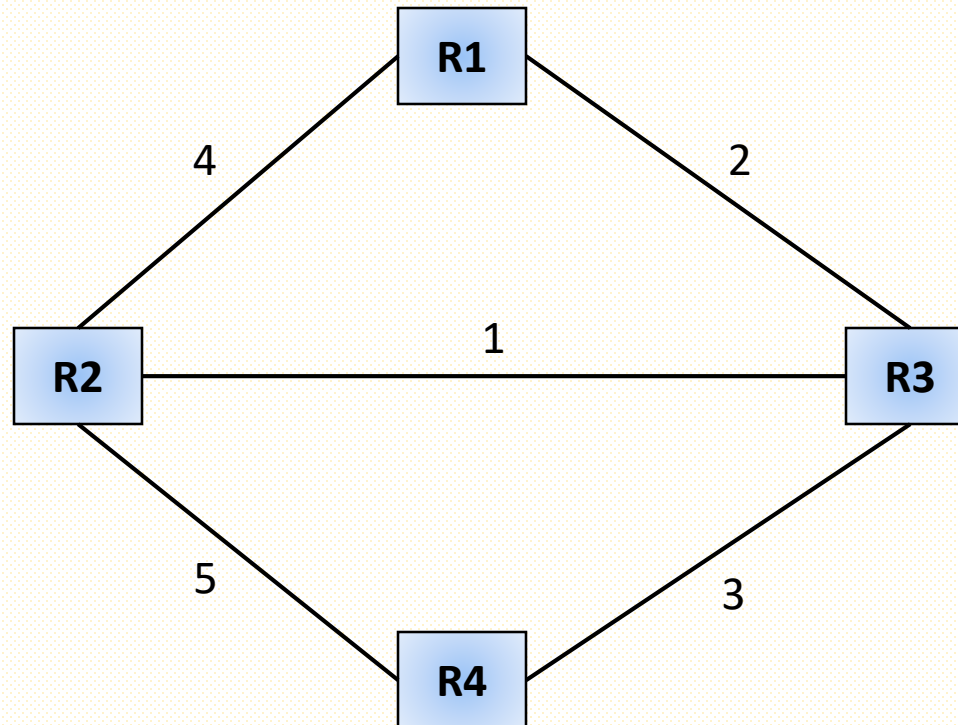


D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	7	R2

Count to ∞ Problem

- ❑ Why this problem $\rightarrow$ because we are only sharing the distance and not **how**. If R2 shares that I can reach R4 via R3 then R3 can immediately identify that its me and there is no valid link.
- ❑ This is done in link-state protocols $\rightarrow$ flood entire network topology.

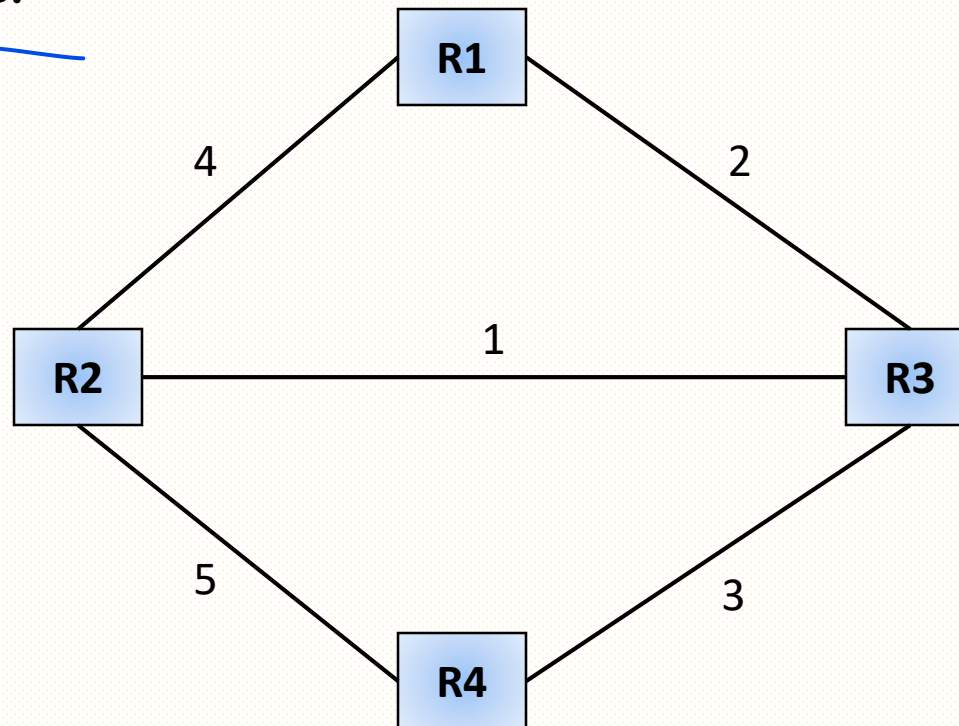
D	C	NH
R1	3	R3
R2	0	--
R3	1	R3
R4	6	R3



D	C	NH
R1	2	R1
R2	1	R2
R3	0	--
R4	7	R2

Link-State Routing

- ❑ Each router discovers its neighbor by hello packet → flood the info to all routers.
- ❑ Now each router have the entire topology.
- ❑ Each router runs Dijkstra's shortest path algorithm locally to compute route.



Link-State Routing

❑ Full routing table: DIY.

❑ Why link state is better:

- No count to infinity problem.
- Converges faster (all routers run in parallel)
- Scales well in large network.

❑ Why RIP?

- Distance Vector → RIP
- Link State → OSPF
- Path Vector → BGP (not always shortest)

D L P